Computación de Alto Rendimiento

Grado en Ingeniería Informática

LABORATORIO EVALUABLE

Laboratorio 1

TermoHPC I — Simulación térmica paralela con OpenMP

Laboratorio · Evaluable · Defensa obligatoria

Docentes: Elam Uceda Herrero · Javier Vázquez Pereda



I. Introducción

Una placa metálica cuadrada se calienta por uno de sus bordes y se mantiene fría por los otros tres. ¿Qué temperatura alcanza cada punto de la placa cuando la situación se estabiliza? La pregunta es de las primeras que resuelve cualquier curso de métodos numéricos, y la respuesta es un método que cabe en cinco líneas: divide la placa en una malla, y repite muchas veces la operación «cada celda toma la media de sus cuatro vecinas» hasta que el campo deje de cambiar.

Ese método —el **esquema de Jacobi**— es el esqueleto de una familia enorme de simulaciones reales: difusión de calor, difusión de contaminantes, campos electrostáticos, flujo en medios porosos, filtrado de imágenes. Se le llama **estarcido** (en inglés *stencil*) porque cada celda se calcula aplicando siempre la misma plantilla de vecinas. Y es, con diferencia, el patrón de cálculo que más horas de supercomputación consume en el mundo.

Es también un problema honesto para aprender computación de alto rendimiento, y por una razón que conviene decir desde la primera página: **escala, pero no escala bien**. Paralelizarlo es fácil —no hay dependencias entre celdas dentro de un paso, así que la directiva sale sola— y sin embargo el speedup que se obtiene está muy por debajo del número de hilos. Entender por qué es exactamente el trabajo de este laboratorio, y es un aprendizaje que no se puede tener con un problema que escala de maravilla.

TermoHPC es además el **hilo conductor de la asignatura**. Aquí lo vais a resolver con OpenMP sobre memoria compartida. En el **Laboratorio 2**, al final del curso, resolveréis el mismo problema con MPI sobre memoria distribuida y con CUDA sobre la GPU, y compararéis los tres caminos sobre una base común. Lo que midáis y entendáis ahora es la referencia con la que se juzgará todo lo demás; por eso merece la pena hacerlo bien.

> **Lo que de verdad se evalúa aquí**
>
> No es que el programa vaya más rápido. Eso lo consigue cualquiera añadiendo una línea.
>
> Es que sepáis **cuánto** debería ir más rápido, **cuánto** va en realidad, y **por qué** hay diferencia entre las dos cosas. Una memoria que presenta un speedup de 2,3 con doce hilos y demuestra con datos de dónde sale ese 2,3 vale mucho más que otra que presenta un 3,0 y no lo explica.

DE DÓNDE VENÍS: LA PRÁCTICA 3

Este laboratorio empieza justo al terminar el Tema 3, y da por hecho lo que hicisteis allí.

Aquí **no se vuelve a explicar** cómo se cronometra, cómo se reparte un bucle ni cómo se comprueba que una paralelización es correcta: eso lo medisteis vosotros en la **Práctica 3**: dependencias, planificaciones y reparto del trabajo entre hilos, `reduction` frente a `critical` y `atomic`, variables compartidas sin querer, falsa compartición y tareas.

Y, muy concretamente, **traed el método**: el `crono.h`, las repeticiones quedándoos con el mínimo, el resultado siempre consumido para que `-O2` no borre el trabajo y la comparación con la versión secuencial. Lo que la Práctica 3 no medía —el techo de escalado de vuestra máquina y su ancho de banda de memoria— se mide aquí, en la fase 2 (apartados IV.C y V.G), y es la referencia contra la que se juzga todo lo que midáis en TermoHPC. Si en la Práctica 3 os quedó algo sin medir, este es el momento de cerrarlo.

II. Objetivos

Al terminar el laboratorio, el grupo debe ser capaz de:

- Implementar en C una simulación de difusión de calor por el esquema de Jacobi, **correcta**, instrumentada y con un criterio de parada explícito, y usarla como **referencia** frente a la que se verifica todo lo demás.
- Paralelizarla con OpenMP **justificando cada decisión**: qué bucle se paraleliza y por qué ese y no otro, qué planificación, qué cláusulas de datos, dónde hacen falta barreras y dónde no.
- Establecer un protocolo de **verificación numérica** frente a la versión secuencial, y aplicarlo a cada variante que se mida.
- Medir la **escalabilidad fuerte** con 1, 2, 4, 8, 12, 16, 20 y 24 hilos, calcular speedup y eficiencia, y **situar lo medido frente al techo real de la máquina**, no frente a un ideal inalcanzable.
- Medir la **escalabilidad débil** y explicar en qué se diferencia de la fuerte y qué pregunta responde cada una.
- Comparar las **planificaciones** de OpenMP sobre este problema y justificar con datos cuál conviene y por qué.
- Aplicar un **método de optimización guiado por medición**: localizar el cuello de botella con datos, cambiar una cosa, volver a medir, y repetirlo al menos dos veces.
- Reconocer cuándo un programa está **limitado por el ancho de banda de memoria** y no por el cálculo, y qué consecuencias tiene eso sobre lo que cabe esperar del paralelismo.
- Escribir una memoria que **razone** y defenderla oralmente, respondiendo por cualquier decisión y por cualquier número.

III. El problema: TermoHPC

III.A · De la placa a la malla

La placa es un cuadrado. Se discretiza en una malla de $N \times N$ celdas, y a cada celda se le asocia un número: su temperatura. La malla se guarda en un único vector de `double` de N^2 elementos, recorrido por filas, de modo que la celda de fila i y columna j ocupa la posición $i \cdot N + j$. Esa decisión de almacenamiento no es un detalle: gobierna toda la discusión de rendimiento que viene después.

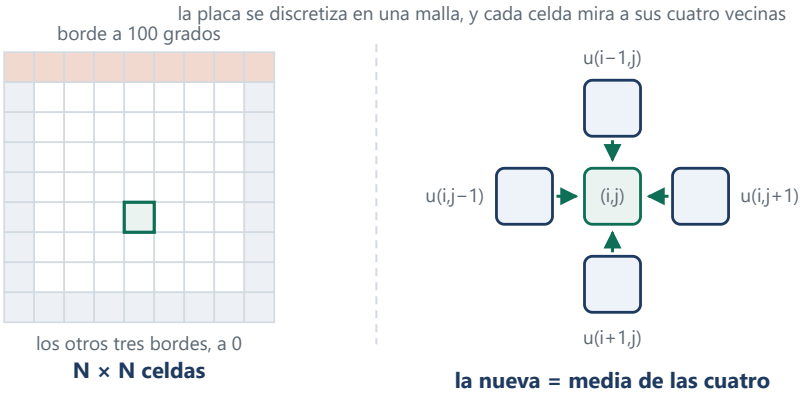


Figura 1.1 · *El esquema de Jacobi en dos dimensiones. La malla discretiza la placa; en cada paso de tiempo el valor nuevo de una celda interior es la media aritmética de sus cuatro vecinas. Las celdas del borde no se calculan: su temperatura está fijada por las condiciones de contorno.*

Las celdas se dividen en dos clases. Las del **borde** —primera y última fila, primera y última columna— tienen la temperatura fijada por las condiciones de contorno y no se calculan nunca. Las **interiores** son las que evolucionan. En cada paso de tiempo, el valor nuevo de una celda interior es la media aritmética de sus cuatro vecinas:

$$u_{i,j}^{t+1} = \frac{1}{4}\left(u_{i-1,j}^{t} + u_{i+1,j}^{t} + u_{i,j-1}^{t} + u_{i,j+1}^{t}\right)$$

Fijaos en el detalle que lo cambia todo: en el lado derecho **todos los valores llevan el superíndice *t***. El paso nuevo se calcula entero a partir del paso viejo. Ninguna celda usa un valor recién calculado en este mismo paso. Esa es la definición del método de Jacobi, y es la propiedad que lo hace paralelizable sin ningún cerrojo: dentro de un paso, las N^2 celdas son independientes entre sí.

III.B · Las condiciones de contorno y el estado inicial

Para que todos los grupos midan lo mismo, el problema queda fijado así:

- **Borde superior** (fila 0): temperatura constante **100**.
- **Los otros tres bordes** (última fila, primera y última columna): temperatura constante **0**.
- **Interior**: temperatura inicial **0** en todas las celdas.

Con esas condiciones el calor entra por arriba y se difunde hacia el interior hasta que el campo se estabiliza. El estado final es único y no depende de cómo se haya llegado a él, lo cual es muy conveniente: cualquier implementación correcta, secuencial o paralela, tiene que converger al mismo campo.

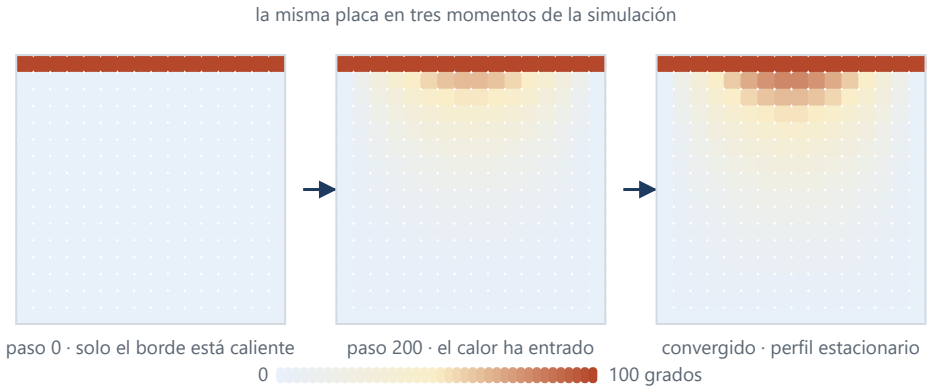


Figura 1.2 · *Evolución de la placa. El calor entra por el borde superior y se difunde hacia el interior; al cabo de suficientes pasos el campo deja de cambiar y se alcanza el régimen estacionario. Ese campo final es el que hay que reproducir, celda a celda, con la versión paralela.*

III.C · Doble buffer: por qué hacen falta dos mallas

La fórmula exige leer el paso viejo y escribir el nuevo. Si se escribiera sobre la misma malla que se está leyendo, al calcular la celda (i, j) las vecinas de arriba y de la izquierda ya habrían sido actualizadas en este mismo paso, y las de abajo y la derecha no. Eso ya no es Jacobi: es otro método (se llama Gauss-Seidel, y converge más deprisa, pero es otro).

Y en paralelo es peor todavía: qué vecinas están actualizadas y cuáles no **depende de cómo se repartan las filas entre los hilos**, así que el resultado cambia con el número de hilos y deja de ser reproducible. Por eso se usan dos mallas y se intercambian los punteros al final de cada paso.



Figura 1.3 · *Por qué Jacobi necesita dos mallas. Si se actualiza sobre la misma malla, una celda puede leer vecinas ya actualizadas en este mismo paso y vecinas todavía sin actualizar, y qué toca a cada una depende de cómo se repartan las filas entre los hilos: el resultado deja de ser reproducible.*

> **LA REGLA QUE NO SE NEGOCIA**
>
> Dos mallas, y al final de cada paso se intercambian **los punteros**, no el contenido. Copiar N^2 `double` de un buffer a otro en cada paso duplica el tráfico de memoria del programa, y en un problema que ya está limitado por la memoria eso se paga entero.

III.D · El criterio de parada

La simulación termina cuando el campo deja de cambiar. Eso se mide con el **residuo**: el mayor cambio que ha sufrido una celda en el paso que acaba de darse.

$$r^{t+1} = \max_{i,j \,\in\, \text{interior}} \left| u_{i,j}^{t+1} - u_{i,j}^{t} \right|$$

Se para cuando r baja de una tolerancia dada, o cuando se alcanza un número máximo de pasos, lo que ocurra antes. En los experimentos de rendimiento se usa siempre **un número de pasos fijo**, porque comparar dos versiones que han dado un número distinto de pasos no compara nada.

> **Dos usos distintos del mismo programa, y no se mezclan**
>
> **Modo simulación**: se corre hasta convergencia con una tolerancia. Sirve para comprobar que el método funciona y para el reto.
>
> **Modo medida**: se corre un número fijo de pasos, sin tolerancia. Es el único modo en el que se cronometra. Todas las tablas de tiempos de la memoria salen de aquí.
>
> ---
>
> Si en una tabla de tiempos dos filas tienen distinto número de pasos, esa tabla no dice nada.

IV. Entorno de trabajo y método de medida

IV.A · Compilación

El laboratorio se desarrolla en C (C11) con OpenMP. Compilad siempre con avisos activados y con optimización:

```
gcc -Wall -Wextra -O2 -std=c11 -fopenmp termo.c -o termo -lm
```

Las tres partes importan. `-Wall -Wextra` porque en este laboratorio un aviso ignorado suele ser un error de verdad. `-O2` porque medir un programa sin optimizar no dice nada sobre su rendimiento. Y `-fopenmp` porque sin él **el programa compila igual**: el compilador ignora las directivas `#pragma` que no entiende, genera un ejecutable secuencial perfectamente correcto, y os pasáis media tarde preguntándoos por qué el speedup es exactamente 1,00.

> **CUIDADO CON `-O2` AL MEDIR**
>
> Si el resultado de un cálculo no se usa para nada, el compilador tiene todo el derecho a eliminar el cálculo entero, y lo hace. Un bucle cronometrado cuyo resultado no se imprime puede tardar cero.
>
> La defensa es sencilla y es obligatoria en este laboratorio: **imprimid siempre una magnitud global que dependa de todo el cálculo** —el checksum de la malla—. Además de evitar que el bucle desaparezca, es lo que os sirve para verificar.

IV.B · Cronometraje

Para medir tiempos usad `omp_get_wtime()`, que devuelve segundos de reloj de pared. No uséis `clock()`: en un programa con varios hilos, `clock()` mide **tiempo de CPU acumulado de todos los hilos**, así que al paralelizar sube en lugar de bajar y produce «speedups» menores que uno que no significan nada.

```
/* Medida limpia de una variante. Devuelve el MEJOR de 'reps' intentos. */
double mide(int n, int pasos, int hilos, int reps, double *checksum)
{
❶     double mejor = 1e30;
      for (int r = 0; r < reps; r++) {
❷         inicializa(u, unueva, n);          /* estado de partida IDENTICO */
          double t0 = omp_get_wtime();
          jacobi(u, unueva, n, pasos, hilos);
❸         double dt = omp_get_wtime() - t0;
          if (dt < mejor) mejor = dt;
          *checksum = suma_interior(u, n);
      }
      return mejor;
}
```

Tres decisiones, y las tres hay que declararlas en la memoria. ❶ Se repite y se toma el **mínimo**, no la media: el ruido del sistema operativo solo puede *añadir* tiempo, nunca quitarlo, así que el mínimo de varias ejecuciones es la estimación más limpia del coste real del programa. Si preferís dar la media, dadla, pero decid cuál dais y sed coherentes en todas las tablas.

❷ La inicialización va **dentro** del bucle de repeticiones y **fuera** del cronómetro. Dentro, porque si no la segunda repetición partiría de una malla ya evolucionada y estaría midiendo otra cosa. Fuera del cronómetro, porque preparar los datos no es simular.

❸ Solo se cronometra el bucle temporal. Ni la reserva de memoria, ni la inicialización, ni el cálculo final del checksum.

IV.C · La máquina de referencia, y por qué no vais a llegar al ideal

Las medidas de este enunciado están hechas en la máquina de referencia de la asignatura: **12 núcleos físicos y 24 hilos lógicos**, con gcc 13.1.0 y `-fopenmp -O2`. `omp_get_max_threads()` devuelve 24.

Y ahora el aviso más importante de este enunciado, porque va a condicionar la lectura de todas vuestras gráficas: **esta máquina no puede dar speedup lineal, y no por culpa de vuestro programa**. Su procesador tiene núcleos de **dos clases**, unos rápidos y otros más lentos y eficientes. Cuando un equipo de hilos se reparte un trabajo a partes iguales, el equipo entero avanza al ritmo del hilo que va más despacio: los rápidos terminan antes y esperan en la barrera.

Eso se puede medir, y es lo primero que conviene medir, porque da la vara con la que se juzga todo lo demás. El experimento es un núcleo puramente aritmético, sin ningún acceso a memoria, en el que **cada hilo hace exactamente el mismo trabajo**. Si la máquina fuese homogénea y hubiese núcleos libres, el tiempo total sería el mismo con 1 hilo que con 12. No lo es:

HILOS	TIEMPO (S)	TRABAJO HECHO	TECHO REAL DE SPEEDUP
1	0,492	1 ×	1,00
2	0,478	2 ×	2,06
4	0,495	4 ×	3,98
8	0,596	8 ×	6,61
12	0,610	12 ×	9,69
16	0,634	16 ×	12,42
20	0,686	20 ×	14,36
24	0,788	24 ×	15,00

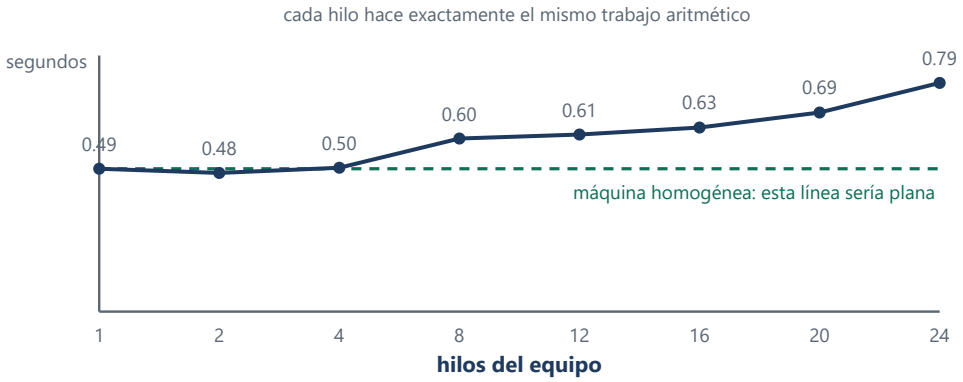


Figura 1.4 · *El techo real de esta máquina, medido. Cada hilo ejecuta un bucle aritmético idéntico y sin accesos a memoria, de modo que el tiempo total debería ser el de un solo hilo mientras queden núcleos libres. No lo es: a partir de cuatro hilos el tiempo sube, porque el procesador tiene núcleos de dos clases y el equipo avanza al ritmo del más lento.*

Cómo se lee esto, y cómo hay que escribirlo en la memoria

La última columna es el **techo real**: el mejor speedup que esta máquina puede dar con ese número de hilos *aunque el programa fuese perfecto*. Cuando publiquéis una curva de speedup, dibujad esa línea además de la diagonal ideal.

La consecuencia práctica: si medís 8,31 con doce hilos y el techo de la máquina son 9,7, no escribáis «el escalado es imperfecto, probablemente por la sobrecarga de OpenMP». Escribid «estamos a un 86 % del máximo que esta máquina permite», que es lo que dicen los datos. Disculparse por no alcanzar un ideal físicamente inalcanzable es un error de análisis, y se penaliza igual que maquillar una tabla.

IV.D · Cómo se fija de verdad el número de hilos

Parece un detalle y arruina barridos enteros. Hay tres formas de decidir cuántos hilos usa una región paralela, y se pisan unas a otras en este orden de prioridad, de menor a mayor: la variable de entorno `OMP_NUM_THREADS`, la llamada `omp_set_num_threads()`, y la cláusula `num_threads()` de la propia directiva.

```
  /* Al principio de main, antes de cualquier region paralela. */
❶ omp_set_dynamic(0);              /* que nadie decida por nosotros */
❷ omp_set_num_threads(hilos);      /* 'hilos' viene de la linea de ordenes */
  ...
  #pragma omp parallel
❸     if (omp_get_thread_num() == 0)
          printf("hilos reales: %d\n", omp_get_num_threads());
```

❶ `omp_set_dynamic(0)` es imprescindible y casi nadie lo pone. Con el ajuste dinámico activado —que suele ser el valor por defecto— la biblioteca puede **darte menos hilos de los que has pedido** si considera que el sistema está cargado. El programa no avisa: simplemente tu fila de «24 hilos» se ejecutó con 16, y tu gráfica se aplana por un motivo que no tiene nada que ver con tu programa.

❷ El número se pasa por la línea de órdenes, no se recompila. Un barrido son ocho ejecuciones del mismo binario.

❸ Y esto es lo que hay que hacer siempre: **preguntar cuántos hilos hay de verdad dentro de la región** y comprobar que coincide con lo que pediste. `omp_get_num_threads()` fuera de una región paralela devuelve 1, que es el error clásico y da la falsa impresión de que el programa no paraleliza.

COMPROBACIÓN OBLIGATORIA ANTES DE MEDIR NADA

Antes de lanzar el barrido del Ejercicio 3, ejecutad una vez con cada número de hilos e imprimid los hilos reales. Si alguna fila no da lo que pedisteis, esa fila no vale. Cuesta un minuto y evita tener que repetir el estudio entero.

IV.E · Tamaños de malla y cuánto tarda esto

Para que dimensionéis los experimentos sin perder tardes enteras, estos son los tiempos de la **versión secuencial** de referencia en la máquina de la asignatura, con 300 pasos:

MALLA N	MEMORIA (2 BUFFERS)	TIEMPO SECUENCIAL (S)	NS POR CELDA Y PASO
512	4 MB	0,05	0,58
1024	16 MB	0,20	0,65
2048	64 MB	1,11	0,88
4096	256 MB	4,16	0,83

Recomendación: usad **N = 2048 y 300 pasos** como configuración principal de todos los estudios de escalabilidad fuerte. Es lo bastante grande para que la malla no quepa en la caché —que es justo lo que hace interesante al problema— y lo bastante pequeña para que un barrido completo de ocho recuentos de hilos con cuatro repeticiones cada uno se haga en pocos minutos. Para el estudio de escalabilidad débil partiréis de una malla menor, porque va a crecer.

ORDEN DE MAGNITUD DE LO QUE VAIS A OBTENER, PARA QUE NADIE SE ASUSTE

En esta máquina, una implementación correcta y razonablemente cuidada de TermoHPC sobre la malla de 2048 da un speedup **del orden de 2 a 3**, y alcanza su máximo con pocos hilos —cuatro u ocho— para después **quedarse plano o incluso empeorar** al seguir añadiendo.

Sí, con doce hilos. No, no es que lo hayáis hecho mal. Es el comportamiento propio de un estarcido limitado por el ancho de banda de memoria, y descubrirlo, medirlo y explicarlo **es el laboratorio**. Si a alguien le sale un speedup de 10 con doce hilos sobre esta malla, lo que hay que revisar no es la suerte: es la verificación.

Y al revés: no os conforméis con anotar «sale 2,3». Ese número hay que explicarlo con el techo de la máquina por un lado y con la cuenta del ancho de banda por otro, que es lo que piden los ejercicios 3 y 6.

V. Fundamentos

Esta sección fija el marco con el que se construye y se juzga el trabajo. Enuncia qué hay que analizar y con qué herramientas; no resuelve ningún ejercicio.

V.A · Verificación numérica: la condición de entrada

Un programa paralelo que va más rápido y da otro número no ha optimizado nada: ha roto el programa. En este laboratorio la verificación no es un apartado opcional del final, es la **condición de entrada** de cualquier medida: una variante que no está verificada no se mide, y si aparece medida en la memoria, esa medida no puntúa.

La comparación se hace siempre contra la **versión secuencial**, ejecutada con la misma malla y el mismo número de pasos, y con dos magnitudes complementarias:

- **La suma total** del interior de la malla (el *checksum*). Es barata, resume el campo entero en un número y detecta cualquier desastre grande. Pero es una suma: dos errores de signo contrario se cancelan, así que por sí sola no basta.
- **La norma del máximo de la diferencia**, celda a celda, entre el campo paralelo y el secuencial. Esta sí es concluyente: si la mayor diferencia individual está por debajo de la tolerancia, no hay ninguna celda mal.

$$\Delta \;=\; \max_{i,j} \left| u_{i,j}^{\text{par}} - u_{i,j}^{\text{sec}} \right| \;\leq\; \varepsilon$$

Por qué hay tolerancia y no igualdad exacta

La pregunta natural es por qué no se exige que los dos campos sean idénticos bit a bit. A veces lo son, y conviene entender cuándo.

El cálculo de una celda —la media de cuatro vecinas— hace siempre las mismas operaciones en el mismo orden, la reparta el hilo que la reparta. Por eso, **si la única diferencia entre secuencial y paralelo es quién calcula qué celda**, el campo resultante suele salir idéntico bit a bit. Es una comprobación excelente y merece la pena buscarla.

Pero en cuanto aparece una **reducción** —el máximo del residuo, o una suma global— la cosa cambia: OpenMP combina los resultados parciales en un orden que depende del número de hilos y del reparto, y la suma en coma flotante **no es asociativa**. Sumar los mismos números en otro orden da, en general, un resultado ligeramente distinto. Por eso la comparación se hace con tolerancia, y por eso hay que declarar cuál se usa y justificarla.

> **Qué tolerancia es razonable**
>
> Con `double` y campos de orden 100, una tolerancia de 10^{-9} es holgada y detecta cualquier error de verdad. Una diferencia de 10^{-12} es reordenación de la coma flotante; una de 10^{-2} es un error de programa.
>
> Lo que **no** vale es fijar la tolerancia después de ver el resultado para que cuadre. La tolerancia se declara antes, con su justificación, y si el programa no la pasa, se arregla el programa.

Los tres errores que caza esta verificación

Vale la pena saber qué se está buscando, porque son los tres fallos que aparecen de verdad:

- **Actualización in situ.** Alguien ha ahorrado un buffer. El campo sale parecido pero distinto, y —la firma característica— **cambia con el número de hilos**.
- **Bordes tocados.** Un bucle que va de 0 a $N{-}1$ en lugar de 1 a $N{-}2$ se come las condiciones de contorno. El campo se enfría entero.

- **Variable compartida que debía ser privada.** Los índices del bucle interior, o cualquier temporal. Da resultados distintos en cada ejecución **con el mismo número de hilos**, que es lo que lo distingue del primer caso.

V.B · Speedup y eficiencia

Con T_1 el tiempo de la versión de referencia y T_p el de la paralela con p hilos:

$$S(p) = \frac{T_1}{T_p} \qquad E(p) = \frac{S(p)}{p}$$

Hay una decisión escondida en T_1 que hay que tomar explícitamente y declarar, porque cambia todos los números:

- **Speedup absoluto**: T_1 es el tiempo de la **mejor versión secuencial**, compilada sin `-fopenmp`. Es el honesto y es el que se pide aquí.
- **Speedup relativo**: T_1 es el tiempo de la **versión paralela ejecutada con un solo hilo**. Es cómodo y siempre sale mejor, porque arrastra la sobrecarga de OpenMP también en el denominador de referencia.

Dad los dos si queréis, pero **etiquetadlos**. Presentar un speedup relativo llamándolo speedup a secas es de los errores que más engordan una gráfica sin que se note.

V.C · Escalabilidad fuerte y escalabilidad débil

Son dos preguntas distintas y hacen falta las dos.

La **escalabilidad fuerte** fija el problema y añade hilos: la malla es siempre de 2048×2048 y se pasa de 1 a 24 hilos. Responde a «tengo esta simulación, ¿cuánto puedo acortar el reloj?». Es la que se degrada antes, porque al repartir un problema fijo entre más hilos a cada uno le toca menos trabajo y las partes que no se reparten pesan cada vez más.

La **escalabilidad débil** fija el trabajo *por hilo* y hace crecer el problema con el número de hilos. Responde a «si me dan una máquina el doble de grande, ¿puedo simular un problema el doble de grande en el mismo tiempo?». Si el programa escalara idealmente, la curva sería plana.



Figura 1.5 · *Escalabilidad fuerte y débil no miden lo mismo. La fuerte fija el problema y pregunta cuánto se acorta el reloj al añadir hilos; la débil fija el trabajo de cada hilo y hace crecer el problema, y pregunta si el reloj se mantiene. Un programa puede ir bien en una y mal en la otra.*

CÓMO SE HACE CRECER LA MALLA EN EL ESTUDIO DÉBIL

El trabajo de un paso es proporcional al **número de celdas**, N^2, no a N. Para que cada hilo conserve el mismo número de celdas al pasar de 1 a p hilos hay que multiplicar N por $\sqrt{p}$, no por p.

Con N_1 = 1024 sale la serie 1024, 1448, 2048, 2896, 3547, 4096, 4579, 5017 para 1, 2, 4, 8, 12, 16, 20 y 24 hilos. Redondear al entero más próximo está bien; **decid en la memoria que lo habéis hecho**, porque introduce una desviación de décimas en el trabajo por hilo.

V.D · Qué bucle se paraleliza, y por qué el de fuera no

El programa tiene tres bucles anidados: el temporal, el de filas y el de columnas.

El **bucle temporal no se puede paralelizar**, y no es una cuestión de eficiencia sino de corrección: el paso t+1 necesita el resultado completo del paso t. Hay una dependencia real entre iteraciones. Ese bucle es secuencial por naturaleza, y es la razón de que haga falta una barrera entre pasos.

El **bucle de filas** es el candidato natural. Sus iteraciones son independientes, y como la malla se guarda por filas, dar a cada hilo un bloque de filas contiguas hace que cada uno recorra una región contigua de memoria.

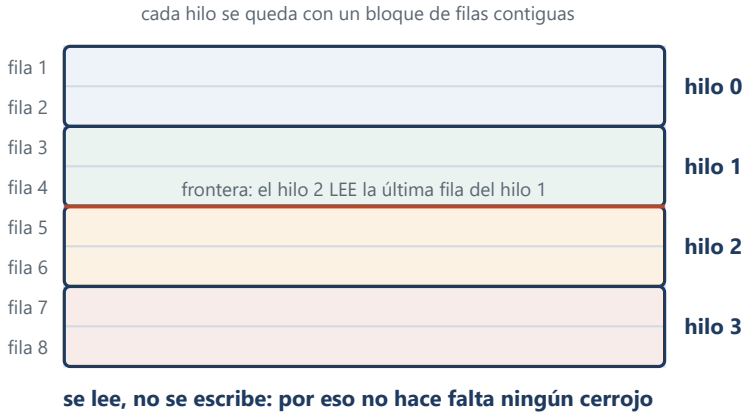


Figura 1.6 · *Descomposición por franjas de filas. Con la malla almacenada por filas, dar a cada hilo un bloque de filas contiguas hace que cada uno recorra memoria contigua. En la frontera entre dos franjas un hilo lee la fila del vecino, pero solo la lee: con doble buffer esa lectura es del buffer antiguo, que nadie está modificando.*

El **bucle de columnas** se podría paralelizar en lugar del de filas, y es instructivo pensar por qué es mala idea: cada hilo se llevaría un conjunto de columnas, es decir, trocitos salteados de todas las filas. Cada línea de caché que trajera contendría datos de varios hilos a la vez, y el rendimiento se desplomaría por falta de localidad y por falsa compartición en la escritura.

La opción `collapse(2)`

OpenMP permite fundir los dos bucles en uno con `collapse(2)`, repartiendo las $(N-2)^2$ celdas entre los hilos. Suena bien —hay muchísimo más paralelismo disponible— y en este problema concreto suele salir **peor**. Merece la pena que lo midáis y que expliquéis el resultado; forma parte del Ejercicio 6.

V.E · Dónde se paga el paralelismo

Hay tres costes, y conviene tenerlos separados porque se atacan de forma distinta.

Uno: abrir y cerrar el equipo de hilos. Una directiva `parallel for` dentro del bucle temporal abre y recoge el equipo en cada uno de los pasos. Cada apertura cuesta del orden de microsegundos; multiplicada por 300 pasos, puede o no ser despreciable frente al trabajo de cada paso. La única forma de saberlo es medirlo.

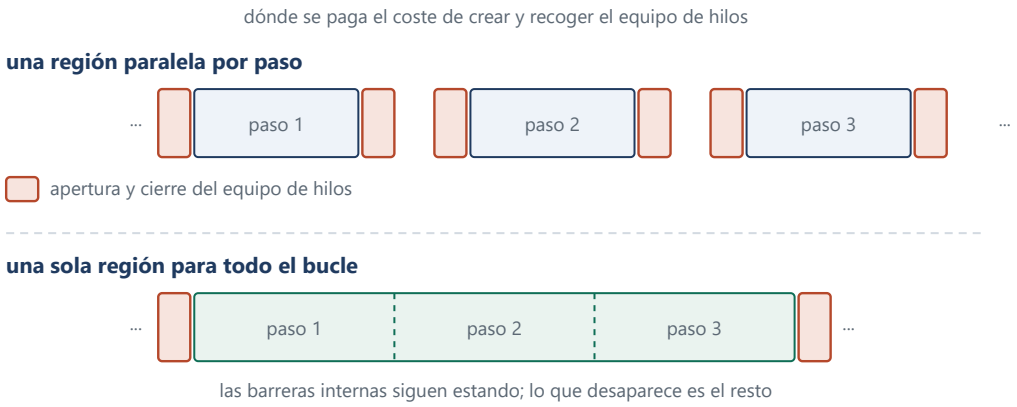


Figura 1.7 · *Las dos formas de organizar el bucle temporal. Con una directiva dentro del bucle se abre y se cierra un equipo de hilos en cada paso; con una sola región paralela que envuelve todo el bucle, el equipo se crea una vez. Las barreras entre pasos son obligatorias en los dos casos, porque un paso no puede empezar antes de que el anterior termine.*

Dos: las barreras. Al final de cada paso todos los hilos tienen que haber terminado antes de que empiece el siguiente. Esa barrera es obligatoria y no se puede quitar; lo que sí se puede es asegurarse de que no haya *más* barreras de las necesarias. Y el coste de una barrera no es el de la barrera en sí: es el **tiempo que los hilos rápidos pasan esperando al más lento**, que en una máquina de núcleos heterogéneos es justo el efecto que midió el techo.

Tres: la memoria. Y este es el grande en este problema. Conviene hacer la cuenta.

V.F · Por qué este problema está limitado por la memoria

Contemos qué hace el programa por cada celda interior que calcula:

- **Cálculo**: tres sumas y una multiplicación. Cuatro operaciones en coma flotante.
- **Memoria**: lee cuatro vecinas y escribe un valor. Con `double`, cinco accesos de 8 bytes, aunque la caché reduce mucho las lecturas efectivas.

La razón entre operaciones aritméticas y bytes movidos —la **intensidad aritmética**— es de las más bajas que se pueden encontrar. Un procesador moderno hace del orden de cien operaciones en coma flotante en el tiempo que tarda en traer un dato de la memoria principal. En un problema así, **el procesador se pasa la vida esperando a la memoria**.

> **La consecuencia, que es la lección central del laboratorio**
>
> Añadir hilos multiplica la capacidad de cálculo, pero **no multiplica el ancho de banda de la memoria**: el bus es uno y se comparte. Cuando un programa está limitado por la memoria, a partir de cierto número de hilos el bus se satura y añadir más hilos deja de aportar. Ahí se aplana la curva de speedup.
>
> Esto no es un defecto de vuestro programa ni de OpenMP. Es una propiedad del problema y de la máquina, y saber reconocerla, medirla y decirla con datos es exactamente lo que se evalúa.

Hay dos consecuencias prácticas que van a aparecer en vuestras medidas y conviene anticipar:

- El speedup depende del **tamaño de la malla**, y no de forma monótona. Una malla grande satura el ancho de banda; una malla pequeña deja tan poco trabajo en cada paso que el coste fijo de repartirlo y sincronizarlo se come la ganancia. Entre los dos extremos hay un tamaño que va mejor que ninguno de ellos. Si medís con un solo N y publicáis el resultado como «el speedup de TermoHPC», estáis describiendo ese N, no el problema.
- **Optimizar la aritmética no sirve de nada.** Si el procesador está esperando a la memoria, ahorrarle una multiplicación no le ahorra tiempo. Las optimizaciones que pueden servir son las que reducen el tráfico de memoria o mejoran la localidad; las que tocan el cálculo, no. Comprobar cuál es cuál con datos propios es

parte del Ejercicio 6.

> **Y un aviso que os va a ahorrar frustración**
>
> «Puede servir» no es «va a servir». Varias de las optimizaciones que suenan obligatorias en un problema como este —trabajar por bloques, sacar la región paralela fuera del bucle temporal, calcular el residuo con menos frecuencia— **en esta máquina y con esta malla no mejoran nada, y algunas empeoran**. Está comprobado.
>
> Eso no las convierte en malas ideas: las convierte en **hipótesis**, que es exactamente lo que el Ejercicio 6 os pide formular y comprobar. Una iteración que se mide y no mejora es un resultado con todas las de la ley, y el enunciado os pide explícitamente que contéis al menos una. Lo que no vale es aplicarlas «porque en los libros salen» y presentarlas como mejoras sin haberlas medido.

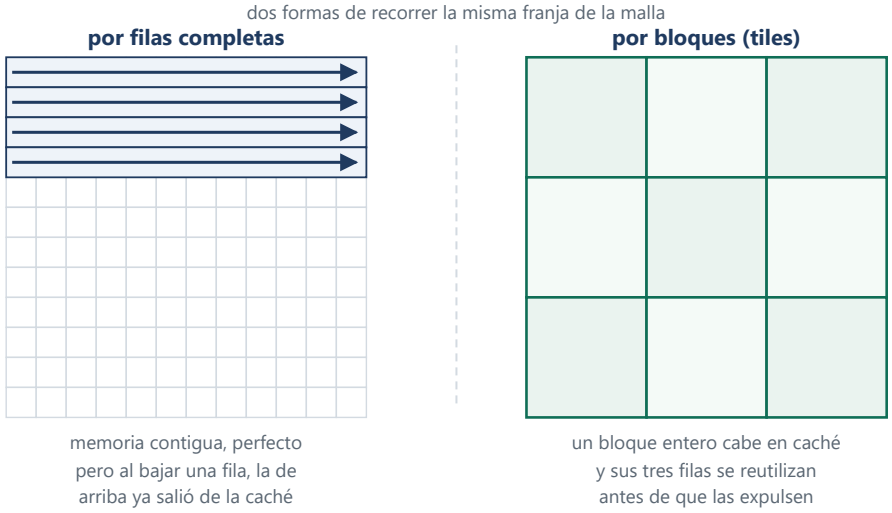


Figura 1.8 · *Recorrido por filas frente a bloqueo por bloques. El estarcido necesita la fila de arriba, la de en medio y la de abajo; recorriendo filas completas de una malla grande, cuando se llega a la fila siguiente la de arriba ya ha sido expulsada de la caché y hay que traerla otra vez. La idea del bloqueo es mantenerla dentro. Si eso compensa el peor recorrido secuencial de la memoria es una pregunta empírica, y la respuesta depende del tamaño de la malla y de la máquina: hay que medirla.*

V.G · Cómo se mide el ancho de banda, y cuál es el de esta máquina

Todo lo anterior es un argumento cualitativo. El Ejercicio 6 pide convertirlo en un número, y merece la pena porque es **el argumento más sólido que se puede llevar a la defensa**: en vez de decir «creemos que está limitado por la memoria», se dice «conseguimos 39 de los 50 GB/s que da esta máquina, luego lo está».

1 · El ancho de banda que consigue un programa

Se calcula dividiendo los **bytes que el programa mueve** entre el **tiempo que tarda en moverlos**:

$$B \;=\; \frac{\text{bytes por paso}}{\text{segundos por paso}}$$

Contar los bytes exige declarar un modelo, y aquí hay que ser explícito porque la cifra depende de él. Para un paso de Jacobi sobre una malla de $N{\times}N$, en el **caso más favorable** —cuando la caché reaprovecha las vecinas, que es lo que ocurre recorriendo por filas—, cada celda interior implica leer su propio valor una vez y escribir el nuevo:

$$\text{bytes por paso} \;\approx\; (N-2)^2 \times (8_{\text{lectura}} + 8_{\text{escritura}}) \;=\; 16\,(N-2)^2$$

Es una **cota inferior** del tráfico real, y conviene decirlo así en la memoria. El tráfico verdadero es mayor por dos motivos: al escribir una línea de caché el procesador suele leerla antes (lo que añadiría 8 bytes más por celda), y en las fronteras entre franjas de hilos hay líneas que viajan dos veces. Dad la cuenta que uséis, con su modelo declarado; lo que no vale es dar un número sin decir cómo se ha contado.

2 · El techo: cuánto ancho de banda da la máquina

Un número de ancho de banda no significa nada sin su referencia. Y la referencia no es la que pone el fabricante en la ficha técnica —esa es la teórica y no se alcanza nunca—, sino la que la máquina entrega de verdad, que se mide igual que se midió el techo de cálculo del apartado IV.C: con un núcleo que no haga otra cosa que mover memoria.

El clásico es el **triad** de STREAM, `a[i] = b[i] + f*c[i]`, sobre vectores tan grandes que no quepan en ninguna caché. Sobre tres vectores de 40 millones de `double` —960 MB en total— la máquina de la asignatura da esto:

HILOS	TIEMPO (S)	GB/S (MODELO DE 24 BYTES)	GB/S (MODELO DE 32 BYTES)
1	0,0248	38,7	51,6
2	0,0191	50,3	67,1
4	0,0196	49,0	65,4
8	0,0210	45,6	60,8
12	0,0226	42,6	56,7
16	0,0225	42,6	56,9
24	0,0227	42,2	56,3

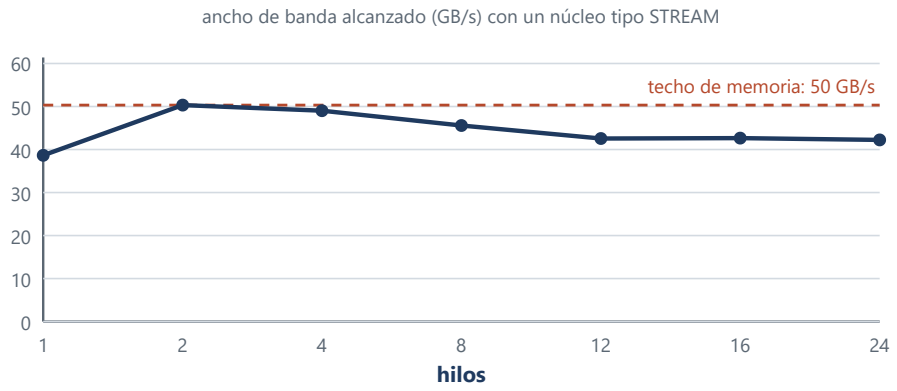


Figura 1.9 · *El ancho de banda de memoria de la máquina, medido. Un solo hilo ya consigue más de las tres cuartas partes del máximo, y el máximo se alcanza con* **dos** *hilos; a partir de ahí, añadir hilos no aporta ancho de banda y llega a restarlo. Esta curva es la explicación cuantitativa de por qué un estarcido como TermoHPC no puede escalar como escala un problema limitado por el cálculo.*

> **Leed esta tabla despacio, porque explica el laboratorio entero**
>
> **Un solo hilo ya consigue 38,7 GB/s**, que es el 77 % del máximo de la máquina. Y el máximo, 50,3 GB/s, se alcanza con **dos** hilos. A partir de ahí la curva no sube: se aplana y luego baja un poco.
>
> Esa es la diferencia de fondo entre los dos tipos de problema de este documento. El cálculo se multiplica al añadir núcleos —de ahí que el filtro espectral llegue a un speedup de dos cifras—. **El ancho de banda no se multiplica**: el bus de memoria es uno y ya estaba casi lleno con un hilo. Un programa cuyo cuello de botella sea la memoria tiene, por tanto, un techo propio de aproximadamente **× 1,30**, y ningún número de hilos lo va a cambiar.

3 · La cuenta hecha, sobre la versión secuencial de referencia

Con los tiempos del apartado IV.E se puede hacer la cuenta completa para la malla principal. Con N = 2048 y 300 pasos, la versión secuencial tarda 1,11 s, es decir 3,70 ms por paso, y mueve como mínimo

$$16 \times 2046^2 \;=\; 67\ \text{MB por paso}$$

de donde sale un ancho de banda conseguido de **18,1 GB/s**, frente a los 38,7 GB/s que da la máquina con un hilo. O sea, la versión secuencial ya está usando alrededor del 47 % del ancho de banda que un hilo puede conseguir, **haciendo además todo el cálculo**.

Esa es la cuenta que hay que repetir en el Ejercicio 6 **para la versión paralela**, y contrastarla con el techo de 50,3 GB/s. El resultado os dirá, con un número y no con una impresión, cuánto margen os queda de verdad. Si os sale cerca del techo, el programa está tan optimizado como este problema permite en esta máquina, y decirlo con la cuenta delante vale más que cualquier optimización adicional.

> **Y DE PASO, UNA COMPROBACIÓN DE COHERENCIA**
>
> Si al hacer esta cuenta os sale un ancho de banda **mayor** que el techo medido de la máquina, no habéis descubierto nada: os habéis equivocado. Las dos causas habituales son contar mal los bytes (olvidar que la malla tiene N^2 celdas y no N) y dividir por el tiempo total en lugar de por el tiempo por paso. Es una comprobación que cuesta diez segundos y que caza errores de un orden de magnitud.

V.H · Las planificaciones

La cláusula `schedule` decide cómo se reparten las iteraciones del bucle entre los hilos:

- `schedule(static)` sin trozo: el rango se parte en p bloques contiguos, uno por hilo. Reparto decidido en compilación, coste de gestión nulo, localidad máxima.
- `schedule(static, c)`: bloques de c iteraciones repartidos por turnos. Sigue siendo estático, pero entrelaza a los hilos.
- `schedule(dynamic, c)`: cada hilo, al terminar su trozo, pide otro. Se adapta al desequilibrio, pero cada petición cuesta y el reparto deja de ser contiguo.
- `schedule(guided)`: como el dinámico, pero los trozos empiezan grandes y van menguando. Un término medio.

La regla que traen los libros dice así: cuando todas las iteraciones cuestan lo mismo, el estático gana, porque no hay desequilibrio que corregir y todo lo que hace el dinámico es añadir gestión y romper la localidad; cuando el coste varía mucho de una iteración a otra, el dinámico compensa de sobra lo que cuesta.

Es una buena primera aproximación y es la que hay que tener en la cabeza. Pero **no es toda la verdad, y en esta máquina se queda corta**: presupone que los núcleos son intercambiables, y aquí no lo son. El RESUELTO 3 lo mide y enseña qué hay que añadirle. Retenedlo, porque afecta a cómo hay que redactar la conclusión del Ejercicio 5: la pregunta no es «¿está equilibrado mi bucle?» sino «¿cuánto desequilibrio hay, de dónde viene, y compensa lo que cuesta corregirlo?».

Y en todo caso, la regla —completa o no— **hay que justificarla con datos propios, no citarla**. Sobre TermoHPC hay un factor más en juego que en cualquiera de los ejemplos, y es el que decide: romper el reparto contiguo de las filas no solo cuesta gestión, cuesta **localidad**, y en un problema limitado por la memoria eso se paga muy caro. Cuál de las dos fuerzas gana es exactamente lo que tenéis que medir.

V.I · Los seis errores que estropean un estudio de rendimiento

Ninguno de estos es hipotético: los seis aparecen todos los años, y cada uno invalida las tablas en las que se ha colado. Repasadlos antes de dar por buena una medida.

ERROR	CÓMO SE RECONOCE	QUÉ HACER
Medir con `clock()`	El «tiempo» sube al añadir hilos y salen speedups menores que 1 en todas las filas.	Usar `omp_get_wtime()`. `clock()` suma el tiempo de todos los hilos.
Compilar sin `-fopenmp`	El speedup es 1,00 clavado en las ocho filas y `omp_get_num_threads()` devuelve 1.	Añadir el flag. Sin él las directivas se ignoran en silencio.
Ajuste dinámico activado	La curva se aplana antes de lo esperable y el número de hilos reales no coincide con el pedido.	`omp_set_dynamic(0)` y comprobar los hilos reales dentro de la región (apartado IV.D).
Cronometrar la inicialización	Las mallas pequeñas salen desproporcionadamente lentas y deforman la tendencia justo donde se lee.	Sacar del cronómetro la reserva de memoria, la inicialización y el checksum.
No reinicializar entre repeticiones	La segunda repetición sale mucho más rápida que la primera, sistemáticamente.	Reinicializar dentro del bucle de repeticiones: si no, se parte de una malla ya evolucionada.
Una sola ejecución por punto	La curva tiene dientes de sierra que cambian de sitio cada vez que se repite el barrido.	Repetir, tomar el mínimo (o la media, pero decid cuál) y declarar cuántas repeticiones.

LA PRUEBA DE HUMO DE TODO EL LABORATORIO

Antes de creerse una tabla, comprobad que la fila de **un hilo** de la versión paralela tarda aproximadamente lo mismo que la versión secuencial. Si tarda mucho más, hay sobrecarga que investigar; si tarda mucho **menos**, no estáis comparando las dos cosas que creéis, y todos los speedups de esa tabla están inflados.

ANTES DE SEGUIR

Los ejemplos resueltos de la sección VI están sobre otro problema, a propósito.

Enseñan el método —medir, verificar, comparar planificaciones y optimizar guiado por datos— sobre un problema distinto de TermoHPC, para que podáis copiar la **forma de trabajar** sin encontraros el laboratorio resuelto. Los números que aparecen en ellos son de ese otro problema y no os sirven como referencia para el vuestro.

VI. Ejemplos resueltos

Los cuatro ejemplos que siguen trabajan sobre problemas auxiliares que **no son TermoHPC**. Los tres primeros usan el mismo:

> **El problema de los resueltos: filtrado espectral de una señal**
>
> Dado un vector x de n muestras, para cada muestra se evalúa una serie truncada y se acumula la energía total:
>
> $$y_i = \sum_{k=1}^{T} \frac{\sin(k\,x_i)}{k} \qquad\qquad E = \sum_{i=0}^{n-1} y_i^2$$
>
> Por cada dato que se lee de memoria se hacen T evaluaciones de un seno, así que este problema está **limitado por el cálculo**, justo al revés que TermoHPC. Esa diferencia es deliberada: verlos escalar de forma distinta enseña más que verlos escalar igual.

El orden es el del método: primero se mide bien (RESUELTO 1), después se verifica (RESUELTO 2), y luego se optimiza guiado por lo medido (RESUELTO 3). El **RESUELTO 4** cambia de problema y de familia: es el que enseña a leer una curva que se estanca, que es la situación en la que os vais a encontrar.

RESUELTO 1 — Medir bien: speedup, eficiencia y el techo de la máquina

Qué se pide: paralelizar el filtro, medir su escalabilidad fuerte de 1 a 24 hilos, calcular speedup y eficiencia, medir también la escalabilidad débil, y leer los resultados frente al techo de la máquina.

1 · La versión secuencial y su paralelización

```
  /* Filtro espectral. Version secuencial de referencia. */
  double filtra_seq(const double *x, double *y, int n, int T)
❶ {
      double E = 0.0;
      for (int i = 0; i < n; i++) {
          double a = 0.0;
❷         for (int k = 1; k <= T; k++)
              a += sin(k * x[i]) / k;
❸         y[i] = a;
          E += a * a;
      }
      return E;
  }
```

El bucle exterior ❶ es el que se paraleliza: sus iteraciones son independientes. El bucle interior ❷ tiene una dependencia real —`a` se acumula de una vuelta a la siguiente— así que no se toca; además, dejarlo secuencial es lo que da a cada iteración exterior un trabajo grande, que es justo lo que interesa. La acumulación de la energía ❸ sí es compartida entre hilos, y es lo único que hay que tratar con cuidado.

```
/* Version paralela. Cambian dos lineas y ninguna mas. */
double filtra_omp(const double *x, double *y, int n, int T)
{
❹    double E = 0.0;
❺    #pragma omp parallel for schedule(static) reduction(+ : E)
    for (int i = 0; i < n; i++) {
        double a = 0.0;                    /* privada: se declara dentro */
        for (int k = 1; k <= T; k++)
            a += sin(k * x[i]) / k;
        y[i] = a;
        E += a * a;
    }
    return E;
}
```

❹ `reduction(+ : E)` da a cada hilo su propio acumulador y los combina al final. Es la forma correcta de acumular; el Ejercicio 6 del Tema 3 ya mostró lo que cuesta hacerlo con `critical`, y el RESUELTO 3 vuelve a medirlo aquí. ❺ La variable `a` se declara **dentro** del bucle, con lo que es privada automáticamente y no hace falta ninguna cláusula. Es la forma más segura de no equivocarse con `private`.

2 · El barrido y la tabla en bruto

Se mide con n = 200.000 y T = 60, cuatro repeticiones por punto, tomando el mínimo, y repitiendo el barrido completo cuatro veces. T_1 es el tiempo de la versión secuencial compilada aparte, es decir, **speedup absoluto**:

HILOS	TIEMPO (S)	SPEEDUP	EFICIENCIA	TECHO DE LA MÁQUINA
1 (secuencial)	0,3755	1,00	100,0 %	1,00
1	0,3749	1,00	100,2 %	1,00
2	0,1974	1,90	95,1 %	2,06
4	0,0999	3,76	93,9 %	3,98
8	0,0653	5,75	71,8 %	6,61
12	0,0452	8,31	69,2 %	9,69
16	0,0370	10,14	63,4 %	12,42
20	0,0313	12,01	60,0 %	14,36
24	0,0294	12,79	53,3 %	15,00

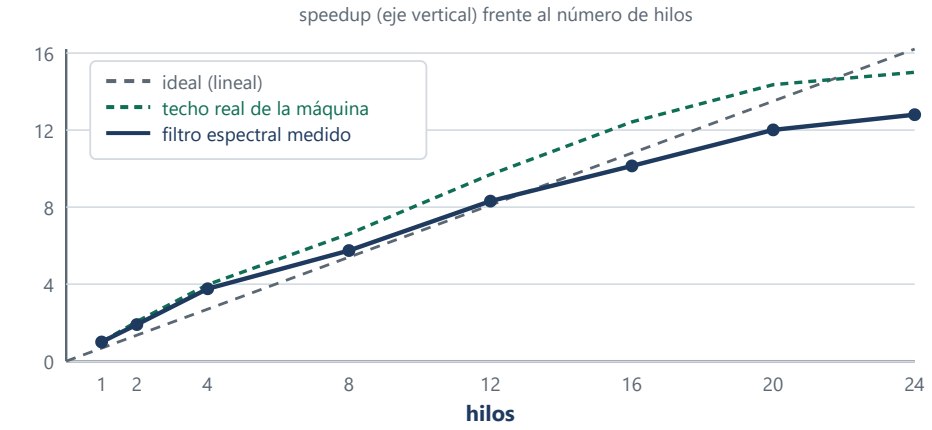


Figura 1.10 · *Escalabilidad fuerte medida. La diagonal es el speedup lineal, que esta máquina no puede alcanzar; la línea verde es el techo que impone su hardware, medido de forma independiente con un núcleo puramente aritmético. La distancia que hay que explicar en la memoria es la que separa la curva azul de la verde, no de la diagonal.*

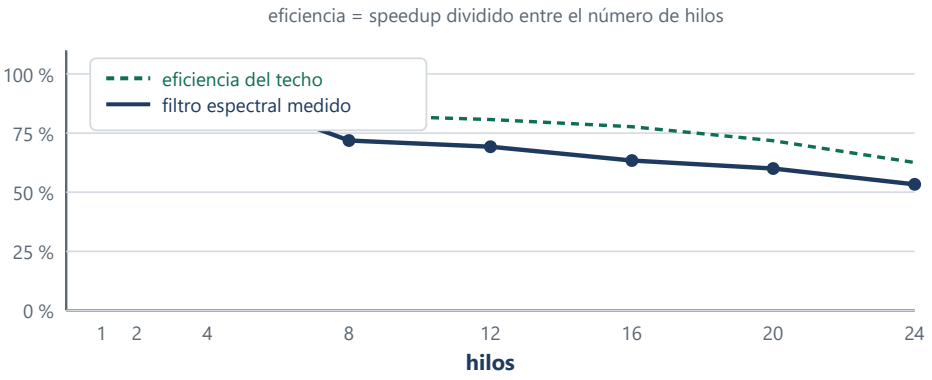


Figura 1.11 · *La misma medida leída como eficiencia. Cae desde el primer momento, y esa caída tiene dos causas superpuestas que hay que separar: la heterogeneidad de la máquina (línea verde) y lo que aporta el propio programa, que es la diferencia entre las dos curvas.*

3 · La lectura, que es donde está la nota

La tabla en bruto no dice nada por sí sola. Lo que se evalúa es esto:

Hasta cuatro hilos, el escalado es prácticamente perfecto y la eficiencia se mantiene muy alta. Coincide con lo que dijo el techo: hasta cuatro hilos todos caben en núcleos de la clase rápida y la máquina se comporta como si fuese homogénea.

De cuatro a doce hilos la eficiencia cae, y la curva se despega de la diagonal ideal. Aquí es donde se separan una memoria buena de una mala. La mala escribe «la eficiencia baja por la sobrecarga de OpenMP». La buena compara con el techo: la curva medida va **pegada a la línea verde**, no a la diagonal, de modo que la caída la explica la heterogeneidad de la máquina y no el programa. Es un problema limitado por el cálculo y sin comunicación entre hilos: no hay ninguna razón para que OpenMP le cueste nada apreciable, y los datos lo confirman.

De 12 a 24 hilos sigue mejorando, y bastante: de 8,31 a 12,79. Conviene entender por qué, porque a primera vista no debería: por encima de 12 no hay núcleos físicos nuevos, y lo que se está usando es el segundo hilo lógico de cada núcleo. La razón es que este núcleo de cálculo está lleno de **dependencias aritméticas** —cada operación espera al resultado de la anterior— y deja huecos en la unidad de coma flotante que el segundo hilo aprovecha. El hiperhilo no duplica nada, pero rellena.

Y sigue siendo la misma historia: el techo también sube en ese tramo, de 9,7 a 15,0, porque se midió con un núcleo que tiene exactamente la misma propiedad. La curva medida no se despega de él. La eficiencia, en cambio, cae —de 69 % a 53 %—, y eso también es esperable: la eficiencia divide entre el número de hilos, y los doce hilos nuevos no son núcleos.

UN AVISO SOBRE LA EFICIENCIA POR ENCIMA DE LOS NÚCLEOS FÍSICOS

A partir de 12 hilos, la eficiencia deja de ser una medida útil de la calidad del programa: está dividiendo entre 24 cuando solo hay 12 núcleos. Una eficiencia del 53 % con 24 hilos no dice que el programa desaproveche la mitad de la máquina; dice que la mitad de esos «hilos» no son procesadores. Presentadla, pero interpretadla con esta salvedad escrita al lado.

LA FRASE QUE HAY QUE SABER ESCRIBIR

«Con doce hilos obtenemos un speedup de 8,31 sobre un techo medido de 9,7, es decir, el 86 % de lo que esta máquina permite. La diferencia que queda no la explica OpenMP: la explica que el equipo avanza al ritmo de sus núcleos más lentos.»

Comparadla con «obtenemos 8,31 en lugar de 12, lo que indica que el escalado no es ideal». Las dos describen exactamente los mismos datos. Solo una demuestra que se han entendido.

4 · La escalabilidad débil del mismo problema

Ahora se fija el trabajo por hilo: n = 25.000 muestras por hilo, de modo que con p hilos se procesan $25.000 \cdot p$ muestras. Aquí el trabajo total es proporcional a n, así que n crece **linealmente** con p; en TermoHPC, donde el trabajo va con N^2, habrá que hacerlo con la raíz.

HILOS	MUESTRAS N	TIEMPO (S)	EFICIENCIA DÉBIL
1	25000	0,0473	100,0 %
2	50000	0,0495	95,6 %
4	100000	0,0504	93,8 %
8	200000	0,0653	72,4 %
12	300000	0,0693	68,3 %
16	400000	0,0710	66,6 %
20	500000	0,0770	61,4 %
24	600000	0,0811	58,4 %

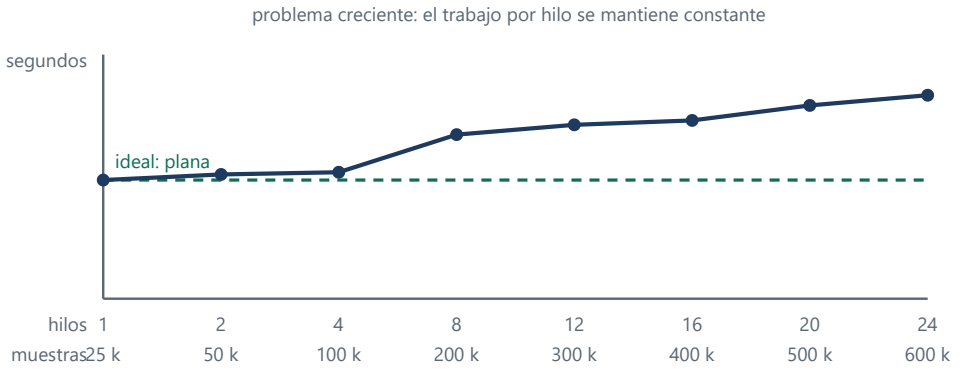


Figura 1.12 · *Escalabilidad débil medida. El tamaño del problema crece con el número de hilos para que cada uno conserve el mismo trabajo. Si el programa escalara perfectamente, la línea sería plana; lo que sube por encima de la referencia es el coste que no se reparte.*

La lectura es la misma y se apoya en el mismo techo. Hasta cuatro hilos la línea es casi plana, que es el comportamiento ideal. A partir de ahí sube, y sube **en la misma proporción** en que el techo se aparta de lo ideal. No hay nada en este problema que se degrade al crecer —no hay comunicación, no hay memoria

compartida caliente—, así que todo lo que sube es la máquina.

> **A QUÉ EJERCICIOS PREPARA**
>
> Al Ejercicio 3 entero (escalabilidad fuerte, speedup, eficiencia y lectura frente al techo) y al Ejercicio 4 (escalabilidad débil). Lo que allí cambia es el problema: TermoHPC está limitado por la memoria, así que sus curvas **no** van a ir pegadas al techo, y tendréis que explicar la diferencia que aquí no existe.

RESUELTO 2 — Verificar: qué se compara, y qué pinta tiene un fallo

Qué se pide: montar la verificación del filtro frente a su referencia secuencial, decidir la tolerancia, y provocar a propósito los dos fallos típicos para ver cómo los caza.

1 · La función de verificación

```
  /* Compara el resultado paralelo con el secuencial. Devuelve 1 si pasa. */
  int verifica(const double *ypar, const double *yseq, int n,
               double Epar, double Eseq, double tol)
  {
      double dmax = 0.0;
❶     for (int i = 0; i < n; i++) {
          double d = fabs(ypar[i] - yseq[i]);
          if (d > dmax) dmax = d;
      }
❷     double dE = fabs(Epar - Eseq) / fabs(Eseq);
      printf("  dif. maxima por muestra: %.3e\n", dmax);
      printf("  dif. relativa de la energia: %.3e\n", dE);
❸     return (dmax <= tol) && (dE <= tol);
  }
```

❶ El bucle recorre **todas** las muestras y se queda con la diferencia mayor, no con la media: una sola celda mal tiene que hacer fallar la verificación, y una media la escondería entre 200.000 valores correctos. ❷ La energía se compara en **términos relativos**, porque es una magnitud grande y una diferencia absoluta de 10^{-9} significa cosas muy distintas según el tamaño del número. ❸ Tienen que pasar las dos comprobaciones: la puntual caza errores localizados, la global caza errores repartidos.

2 · Qué sale cuando todo está bien

Ejecutando la versión correcta con 1, 12 y 24 hilos y comparando siempre contra la secuencial:

HILOS	DIFERENCIA MÁXIMA POR MUESTRA	DIFERENCIA RELATIVA DE LA ENERGÍA	ENERGÍA E
1	0,000000 · 10^+00	0	1,88104432582476e+05
12	0,000000 · 10^+00	1.717e-14	1,88104432582473e+05
24	0,000000 · 10^+00	1.578e-14	1,88104432582473e+05

Dos cosas que merece la pena mirar. La primera: el vector `y` sale **idéntico bit a bit** en los tres casos, con diferencia exactamente cero. Es lo que anticipaba el apartado V.A: cada `y[i]` se calcula con las mismas operaciones en el mismo orden, lo reparta el hilo que lo reparta.

La segunda: la energía **no** sale idéntica, y cambia con el número de hilos. Es la reducción: con 12 hilos se suman 12 parciales, con 24 se suman 24, y en otro orden. La diferencia relativa que se mide es de $1{,}7 \cdot 10^{-14}$, que es el orden de la precisión del `double` arrastrada a lo largo de 200.000 sumas. Eso no es un error: es la coma flotante, y la memoria tiene que decirlo así en lugar de esconderlo.

3 · Fallo provocado nº 1: la variable que debía ser privada

Se saca la temporal `a` fuera del bucle, que es el descuido más común al paralelizar un bucle con acumulador interno:

```
  /* MAL a proposito: 'a' es compartida entre todos los hilos. */
  double E = 0.0, a;
❹ #pragma omp parallel for schedule(static) reduction(+ : E)
  for (int i = 0; i < n; i++) {
      a = 0.0;
      for (int k = 1; k <= T; k++)
          a += sin(k * x[i]) / k;
      y[i] = a;
      E += a * a;
  }
```

❹ Al declararse fuera de la región, `a` es **compartida**: los hilos se pisan el acumulador unos a otros mientras lo van construyendo. Es una condición de carrera de manual, y el programa es incorrecto.

Y ahora viene lo que este resuelto quiere enseñar de verdad, porque no es lo que uno espera. Al ejecutarlo compilado con `-O2`, la verificación **lo da por bueno**: diferencia máxima exactamente cero, dos veces seguidas.

```
$ gcc -Wall -Wextra -O2 -fopenmp verif.c -o verif -lm
$ ./verif 200000 60
sin private      hilos=12  dmax=0.000000e+00  dE_rel=1.701938e-14
sin private      hilos=12  dmax=0.000000e+00  dE_rel=1.717410e-14
```

El mismo programa compilado con `-O0`, en cambio, falla estrepitosamente, y además **falla distinto en cada ejecución**:

```
$ gcc -Wall -Wextra -O0 -fopenmp verif.c -o verif0 -lm
$ ./verif0 200000 60
sin private      hilos=12  dmax=7.301464e+00  dE_rel=5.120104e-01
sin private      hilos=12  dmax=8.705650e+00  dE_rel=5.094679e-01
```

La explicación es que con optimización el compilador se da cuenta de que dentro del cuerpo del bucle nadie sincroniza nada, y mantiene `a` **en un registro de la CPU** durante toda la iteración, volcándola a memoria solo al final. Cada hilo tiene sus propios registros, así que en la práctica cada uno acaba trabajando sobre su copia y la carrera no llega a observarse. Sin optimización, `a` vive en memoria y se lee y escribe de verdad en cada vuelta: entonces sí se pisan.

> **La lección, que vale para todo el laboratorio**
>
> **Que un programa con una condición de carrera dé el resultado correcto no demuestra absolutamente nada.** Puede darlo con este compilador, con esta versión, con este nivel de optimización, con este número de hilos y hoy. El programa sigue siendo incorrecto, y basta cambiar cualquiera de esas cinco cosas para que deje de darlo.
>
> De aquí salen dos consecuencias prácticas. La primera: la corrección de un programa paralelo **se razona sobre el código** —qué variables son compartidas y quién escribe en ellas—, y la verificación numérica sirve para cazar errores, no para certificar que no los hay. La segunda: una verificación que pasa no os autoriza a saltaros la justificación de las cláusulas de datos que pide el Ejercicio 2.

> **LA DEFENSA QUE EVITA ESTE FALLO ENTERO**
>
> Añadid `default(none)` a la directiva. Obliga a declarar explícitamente el reparto de **todas** las variables, y este programa deja de compilar hasta que se decide qué es `a`. Cuesta una palabra, convierte un error silencioso en un error de compilación, y es la única de las tres defensas que no depende de tener suerte con el compilador. Por eso es **obligatoria** en el Ejercicio 2.

4 · Fallo provocado nº 2: la reducción que se olvida

Se quita `reduction(+ : E)` y se deja el resto igual. El resultado es una condición de carrera clásica sobre `E`: cada hilo lee, suma y escribe, y las escrituras se pierden unas a otras. Aquí la carrera **sí** se observa con `-O2`, porque `E` se acumula a lo largo de todo el bucle y el compilador no puede mantenerla en un registro privado sin cambiar el significado del programa:

```
sin reduction    hilos=12  dmax=0.000000e+00  dE_rel=9.610297e-01  E=7.330477794593605e+03
sin reduction    hilos=12  dmax=0.000000e+00  dE_rel=8.536012e-01  E=2.75382548594351e+04
```

La energía correcta es $1{,}881 \cdot 10^5$. Las dos ejecuciones dan $7{,}3 \cdot 10^3$ y $2{,}75 \cdot 10^4$: se ha perdido más del 85 % de las sumas, y se pierde una cantidad distinta cada vez. En cambio el vector `y` sale **perfecto**, con diferencia exactamente cero, porque cada `y[i]` lo escribe un único hilo.

Este caso es el que justifica que la verificación compruebe las **dos** magnitudes. Un protocolo que solo mirase el campo daría este programa por bueno; uno que solo mirase el checksum no distinguiría un error repartido de uno localizado.

> **LOS DOS FALLOS, UNO AL LADO DEL OTRO**
>
> Fijaos en que los dos son condiciones de carrera y sin embargo se comportan al revés. El primero es **invisible con optimización y visible sin ella**; el segundo, **visible siempre**. La diferencia está en si el compilador puede o no quedarse la variable en un registro. Esa asimetría es exactamente la razón por la que la corrección no se puede delegar en «lo he ejecutado y sale bien».

> **A QUÉ EJERCICIOS PREPARA**
>
> Al Ejercicio 2, cuya verificación es condición de entrada de todo lo demás. En TermoHPC las dos magnitudes son el **campo completo** (norma del máximo de la diferencia) y el **checksum**, y el fallo específico que hay que provocar y cazar es el de la actualización in situ, que aquí no aparece porque este problema no tiene doble buffer.

RESUELTO 3 — Optimizar guiado por la medición, en dos iteraciones

*Qué se pide: partir de una versión paralela que funciona pero va mal, localizar el cuello de botella con datos, cambiar **una sola cosa**, volver a medir, y repetirlo.*

Se cambia el régimen del problema para que el cuello de botella no sea el cálculo: ahora n = 4.000 muestras, T = 6, y el filtro se aplica **400 veces seguidas**. El trabajo total es parecido, pero repartido en muchas pasadas cortas: es el régimen en el que la sobrecarga de la paralelización se ve.

Punto de partida

La versión de la que se parte usa `critical` para acumular la energía, que es lo primero que se le ocurre a cualquiera que no conozca `reduction`:

```
  /* Punto de partida: correcto, pero lento. */
  for (int q = 0; q < 400; q++) {
      E = 0.0;
      #pragma omp parallel for schedule(static)
      for (int i = 0; i < n; i++) {
❶         y[i] = muestra(x[i], T);
          #pragma omp critical
          { E += y[i] * y[i]; }
      }
  }
```

❶ Cada una de las *n* iteraciones entra en una sección crítica. Es correcto —la verificación pasa— y es desastroso: la sección crítica serializa por completo la parte de la acumulación, y además hace que los hilos se peleen por el mismo cerrojo.

Iteración 1 · medir, localizar, cambiar una cosa

Lo primero es medir, no suponer. Con 12 hilos, esta versión tarda 0,393 s frente a los 0,287 s de la secuencial: **es más lenta que no paralelizar nada**. Un speedup de 0,73 es un diagnóstico en sí mismo: no es que el paralelismo rinda poco, es que hay algo que serializa y encima cuesta.

El cambio, uno solo: sustituir la sección crítica por una reducción.

```
  /* Iteracion 1: la seccion critica se sustituye por una reduccion. */
  for (int q = 0; q < 400; q++) {
      E = 0.0;
❷     #pragma omp parallel for schedule(static) reduction(+ : E)
      for (int i = 0; i < n; i++) {
          y[i] = muestra(x[i], T);
          E += y[i] * y[i];
      }
  }
```

❷ Ahora cada hilo acumula en su propia copia y OpenMP las combina al final del bucle, en tiempo logarítmico y sin cerrojos. Se vuelve a medir: 0,064 s, es decir, un speedup de 4,45 frente a la secuencial. La verificación sigue pasando.

Iteración 2 · volver a medir, y atacar lo que ahora manda

Con la sección crítica fuera, ¿dónde está lo que falta? El speedup es 4,45 con doce hilos, muy por debajo del 8,31 que daba el mismo problema en el RESUELTO 1. La diferencia entre los dos montajes es que aquí hay **400 pasadas cortas** en lugar de una larga, y por tanto 400 aperturas y cierres del equipo de hilos.

Esa es la hipótesis, y se puede comprobar cambiando una sola cosa: sacar la región paralela fuera del bucle de pasadas.

```
  /* Iteracion 2: UNA sola region paralela para las 400 pasadas. */
❸ #pragma omp parallel
  {
      for (int q = 0; q < 400; q++) {
❹         #pragma omp single
❺         { E = 0.0; }
          #pragma omp for schedule(static) reduction(+ : E)
          for (int i = 0; i < n; i++) {
              y[i] = muestra(x[i], T);
              E += y[i] * y[i];
          }
      }
  }
```

❸ La región paralela envuelve ahora el bucle de pasadas: el equipo se crea una vez, no 400. ❹ La puesta a cero de `E` la hace un solo hilo, y `single` lleva una barrera implícita a la salida que garantiza que nadie empiece a acumular antes de que se haya puesto a cero. ❺ El `for` ya no lleva `parallel`: reparte el trabajo del equipo que ya existe, y su barrera implícita de salida sigue separando una pasada de la siguiente.

Se mide otra vez: 0,085 s, speedup 3,37. La tabla completa del experimento:

VARIANTE (12 HILOS)	TIEMPO (S)	SPEEDUP	MEJORA SOBRE LA ANTERIOR
Secuencial (referencia)	0,2867	1,00	—
Punto de partida · `critical`	0,3927	0,73	—
Iteración 1 · `reduction`	0,0645	4,45	× 6,09
Iteración 2 · región persistente	0,0851	3,37	× 0,76

Qué enseña esto, y qué NO enseña

Enseña el método: no se optimiza lo que uno cree que es lento, se mide, se cambia **una sola cosa**, y se vuelve a medir para comprobar si la hipótesis era buena. Dos iteraciones, dos hipótesis, dos comprobaciones.

Y enseña algo más incómodo: la segunda iteración aporta **mucho menos que la primera**. Eso también es un resultado, y se escribe tal cual. La tentación es presentar solo las optimizaciones que salieron bien; una memoria que cuenta una hipótesis razonable que la medida no confirmó demuestra más criterio que otra que solo enseña aciertos.

Lo que este resuelto **no** enseña es cuál es el cuello de botella de TermoHPC. Es otro problema, con otro régimen y otro cuello. Copiar de aquí las conclusiones en vez del método se nota inmediatamente en la defensa.

El experimento de las planificaciones

Para que la comparación tenga algo que enseñar hacen falta **dos** cargas, no una. Se mide el filtro en sus dos variantes: la **plana**, en la que todas las muestras cuestan lo mismo, y la **triangular**, en la que el número de términos crece linealmente con el índice, de modo que la última muestra cuesta el doble de la media y la primera casi nada. Las dos con 12 hilos y las mismas ocho cláusulas:

PLANIFICACIÓN	TRIANG.: T (S)	SPEEDUP	PLANA: T (S)	SPEEDUP	GANA (TRIANG.)	GANA (PLANA)
schedule(static)	0,0673	5,81	0,0452	8,31	0 %	0 %
schedule(static, 256)	0,0477	8,19	0,0450	8,35	41 %	0 %
schedule(static, 2048)	0,0462	8,45	0,0448	8,39	46 %	1 %
schedule(dynamic, 1)	0,0427	9,16	0,0415	9,06	58 %	9 %
schedule(dynamic, 16)	0,0407	9,61	0,0407	9,23	65 %	11 %
schedule(dynamic, 256)	0,0415	9,41	0,0405	9,27	62 %	12 %
schedule(dynamic, 2048)	0,0441	8,86	0,0414	9,06	53 %	9 %
schedule(guided)	0,0404	9,66	0,0398	9,44	66 %	14 %

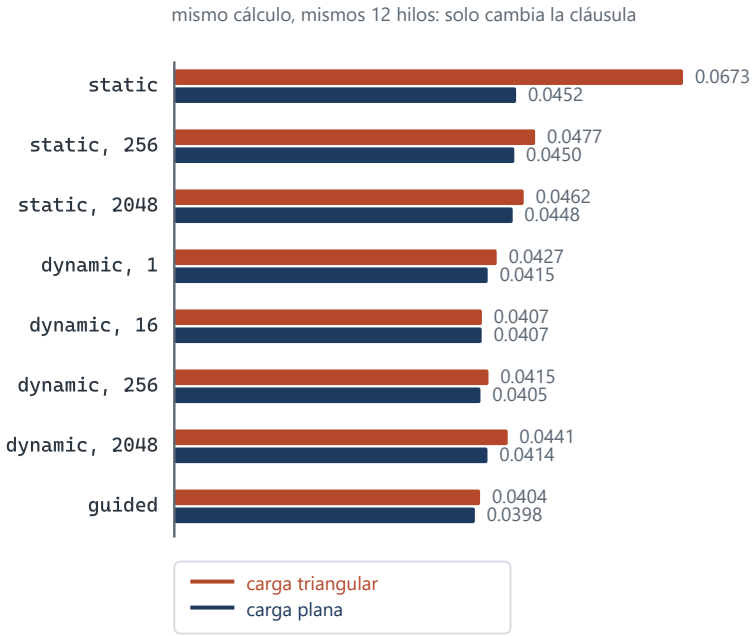


Figura 1.13 · *Las planificaciones comparadas sobre las dos cargas. Con la carga triangular las diferencias son enormes y los repartos dinámicos ganan con holgura, que es lo que dice el libro. Con la carga plana las diferencias se estrechan mucho —pero no desaparecen, y el orden tampoco se invierte: esa es la parte que el libro no cuenta y que hay que explicar.*

La primera lectura es la esperada. Con **carga triangular**, el reparto estático por bloques es el peor de todos, y por una razón que se ve sin medir: el hilo que recibe el último bloque de índices tiene mucho más trabajo que el que recibe el primero, y los demás lo esperan en la barrera. Los repartos dinámicos corrigen el desequilibrio y ganan con holgura. Hasta aquí, el libro.

El resultado que no dice el libro

La segunda lectura es la interesante, y conviene mirarla despacio porque contradice lo que casi todo el mundo espera. La regla de manual dice que en un bucle **equilibrado** el estático debería ganar, porque no hay desequilibrio que corregir y el dinámico solo añade gestión. Y sin embargo, sobre la carga plana, **los repartos dinámicos siguen ganando**, aunque por un margen mucho menor.

La explicación está en el apartado IV.C y es la misma de siempre en esta máquina: **un bucle equilibrado no produce un reparto equilibrado en el tiempo si los núcleos no son iguales**. Con `schedule(static)`, los doce hilos reciben exactamente el mismo número de iteraciones; los que caen en núcleos rápidos terminan antes y se quedan esperando en la barrera a los que caen en núcleos lentos. El reparto es justo en iteraciones e injusto en tiempo.

El reparto dinámico arregla justamente eso: no reparte iteraciones a partes iguales, sino que va dando trabajo **a quien se queda libre**, con lo que los núcleos rápidos acaban procesando más muestras que los lentos y todos terminan a la vez. Corrige un desequilibrio que no está en el problema sino en la máquina.

> **La conclusión, que es más fina que la regla**
>
> La regla de manual —«bucle equilibrado, reparto estático»— presupone que los núcleos son intercambiables. Cuando no lo son, hay que reformularla: el reparto dinámico compensa **cualquier** fuente de desequilibrio, venga del problema o del hardware, y lo cobra en gestión y en localidad.
>
> De modo que la pregunta correcta no es «¿está equilibrado mi bucle?», sino **«¿cuánto desequilibrio hay, de dónde viene, y compensa lo que cuesta corregirlo?»**. Y esa pregunta solo se responde midiendo.
>
> ---
>
> Aviso para el Ejercicio 5: sobre TermoHPC la respuesta **no** es la misma que aquí, y el motivo es que allí lo que se paga por romper el reparto contiguo es mucho más caro. Copiar esta conclusión sin medir se detecta a la primera.

Fijaos también en un detalle que aparece en las dos cargas: `dynamic` con trozos de **una** iteración no es el mejor dinámico. Reparte con la máxima finura, sí, pero paga una petición por iteración. El óptimo está en un trozo intermedio, y encontrarlo es parte del trabajo.

> **A QUÉ EJERCICIOS PREPARA**
>
> Al Ejercicio 5 (planificaciones) y al Ejercicio 6 (optimización guiada por medición), con el guion completo: hipótesis, un solo cambio, nueva medida, y la comparación honesta de lo que aportó cada iteración.

RESUELTO 4 — Leer una curva que se estanca

Qué se pide: medir un problema limitado por la memoria, comprobar que su curva de speedup se para muy pronto, y escribir la explicación con números en vez de con excusas.

Los tres resueltos anteriores trabajan sobre un problema limitado por el cálculo, que escala bien. Este cambia de familia a propósito, porque **la curva que os vais a encontrar en TermoHPC se parece a la de aquí y no a la de allí**, y conviene haber visto antes cómo se razona.

El problema es el **triad** del apartado V.G: `a[i] = b[i] + f*c[i]` sobre tres vectores de 40 millones de `double`. Una multiplicación y una suma por elemento, y tres accesos a memoria. Es el extremo opuesto del filtro espectral: allí había 60 senos por cada dato leído, aquí hay dos operaciones.

1 · La medida

HILOS	TIEMPO (S)	SPEEDUP	GB/S CONSEGUIDOS
1	0,02483	1,00	38,7
2	0,01907	1,30	50,3
4	0,01958	1,27	49,0
8	0,02105	1,18	45,6
12	0,02256	1,10	42,6
16	0,02251	1,10	42,6
24	0,02272	1,09	42,2

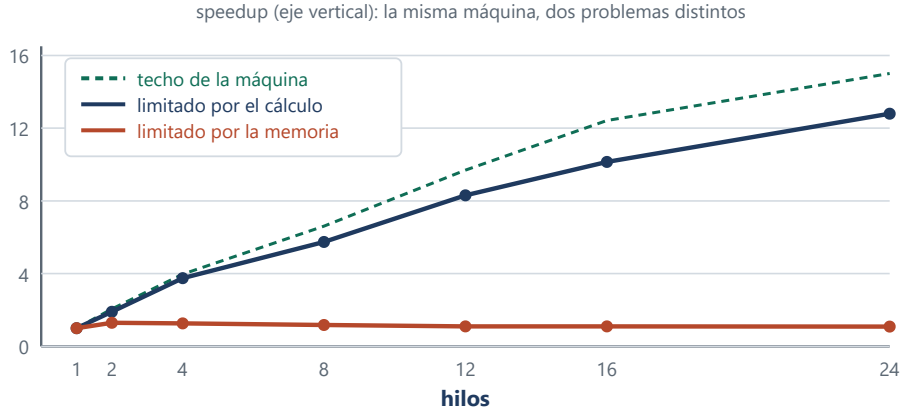


Figura 1.14 · *Las dos familias de problema, medidas en la misma máquina y dibujadas en la misma escala. Arriba, un cálculo intenso sobre pocos datos: sigue al techo del hardware. Abajo, un recorrido de memoria con poco cálculo: se estanca casi de inmediato y no se mueve de ahí. TermoHPC pertenece a la segunda familia, y por eso su curva no se parece a la primera por mucho que se optimice el código.*

2 · Lo que NO hay que escribir

«El speedup máximo obtenido es de 1,30 con dos hilos, muy por debajo del ideal. La escalabilidad es pobre, probablemente debido a la sobrecarga de creación de hilos y a la sincronización de OpenMP.»

Es lo que se escribe cuando no se ha entendido, y tiene tres cosas mal. La primera, «probablemente»: no hay que suponer, hay que medir. La segunda, culpa a OpenMP de algo que OpenMP no ha hecho: este bucle no tiene reducciones, ni secciones críticas, ni desequilibrio, y una única región paralela sobre 40 millones de iteraciones amortiza su coste de apertura hasta hacerlo invisible. La tercera, y la más grave: **presenta como un fallo del programa lo que es una propiedad del problema**.

3 · Lo que sí hay que escribir

La explicación, con la cuenta delante

«El núcleo mueve 24 bytes por iteración y hace dos operaciones en coma flotante: su intensidad aritmética es de 0,08 operaciones por byte, de las más bajas posibles. El programa no está esperando al procesador, está esperando a la memoria.

La medida lo confirma: con un solo hilo ya se alcanzan 38,7 GB/s, y el máximo que la máquina entrega, medido con este mismo núcleo, son 50,3 GB/s con dos hilos. El cociente entre ambos, **× 1,30**, es el speedup máximo que este problema puede dar en esta máquina, y es exactamente el que medimos: 1,30.

Por tanto no estamos lejos del máximo alcanzable: estamos **en** el máximo alcanzable. Añadir hilos por encima de dos no aporta ancho de banda, y de hecho resta un poco, porque más hilos compitiendo por el mismo controlador de memoria empeoran la localidad de los accesos.»

Fijaos en la diferencia de fondo entre las dos redacciones. La primera compara con 24 —un número que no significa nada aquí— y atribuye la diferencia a una causa que no ha comprobado. La segunda **calcula cuál es el máximo posible** y comprueba que se ha llegado a él. La segunda es, además, mucho más corta de defender: no hay nada que optimizar y se puede demostrar por qué.

A QUÉ EJERCICIOS PREPARA

Al Ejercicio 3, y muy especialmente al Ejercicio 6, cuya última exigencia es justamente esta cuenta aplicada a TermoHPC.

Un aviso para que no se copie mal: TermoHPC **no** es un triad. Su estarcido reutiliza cada celda como vecina de cuatro celdas más, así que la caché le ahorra buena parte del tráfico y su techo **no** es el mismo × 1,30 de aquí; es mayor, y tenéis que calcular cuánto. Lo que se traslada de este resuelto es el método —contar bytes, medir el techo, comparar— y no el número.

VII. Ejercicios

El entregable es la memoria descrita en la sección VIII, que desarrolla los seis ejercicios siguientes. **Suman 10 puntos.** Cada requisito va marcado como **[PROG]** (programación y medida) o **[RAZ]** (razonamiento y justificación).

Todo ejercicio debe incluir el desarrollo, **su justificación** y la evidencia real —salidas, tablas, gráficas— que lo demuestra. **Un ejercicio correcto pero sin justificar se califica, como máximo, con la mitad de su puntuación.** No basta con que el programa funcione y sea rápido: hay que saber decir por qué va exactamente lo que va.

Ejercicio 1 (1,5 puntos) — La versión secuencial de referencia

Implementad en C la simulación completa en su versión secuencial. Esta versión es la **referencia**: define lo que es correcto y define T_1. Merece que esté bien hecha.

- **[PROG]** Programa modular: una función de inicialización, una que ejecuta un paso de Jacobi, una que calcula el residuo, una que calcula el checksum y un `main` que lo orquesta. Los parámetros —N, número de pasos, tolerancia, modo— se pasan por línea de órdenes, nunca recompilando.
- **[PROG]** Malla en un **único vector** de `double` recorrido por filas, con reserva dinámica. Nada de arrays de punteros a filas: cambian el patrón de acceso a memoria y falsean todo el estudio posterior.
- **[PROG] Doble buffer con intercambio de punteros**, sin copiar la malla en ningún momento.
- **[PROG]** Los dos modos del apartado III.D: parada por tolerancia y parada por número fijo de pasos.
- **[PROG]** Instrumentación: el programa imprime pasos ejecutados, residuo final, checksum del interior y tiempo, en una línea que se pueda procesar después.
- **[PROG]** Comprobaciones de que la simulación es correcta: que los bordes conservan su valor tras cualquier número de pasos, que el residuo **decrece monótonamente**, y que ninguna temperatura interior queda fuera del intervalo [0, 100]. Las tres son consecuencias físicas del problema y las tres cazan errores reales.
- **[PROG]** Tabla de tiempos secuenciales para N = 512, 1024, 2048 y 4096 con 300 pasos, indicando repeticiones y criterio (mínimo o media).
- **[RAZ]** Comprobad que el tiempo crece con N^2: dividid el tiempo por N^2·pasos y comprobad si el cociente se estabiliza. Si **no** se estabiliza, explicad por qué; la respuesta tiene que ver con qué tamaños de malla caben en la caché, y es un hallazgo, no un fallo.
- **[RAZ]** Justificad por qué el bucle temporal no se puede paralelizar. La respuesta correcta no es «porque va muy rápido»: es una dependencia entre iteraciones y hay que enunciarla.

Ejercicio 2 (2,0 puntos) — Paralelización con OpenMP y verificación numérica

Paralelizad la simulación con OpenMP y montad el protocolo de verificación. Este ejercicio es la puerta de entrada de todos los demás: **si la verificación no pasa, los ejercicios 3 a 6 no se evalúan.**

- **[PROG]** Paralelización del bucle de filas con `#pragma omp parallel for`, usando `default(none)` y declarando explícitamente el reparto de cada variable.
- **[PROG]** El residuo, calculado con `reduction(max : r)`. Nada de `critical` por iteración.
- **[PROG]** Función de verificación que compara el campo paralelo con el secuencial **celda a celda** y devuelve la norma del máximo de la diferencia, además de comparar los checksums.
- **[PROG]** Tabla de verificación con 1, 2, 4, 8, 12, 16, 20 y 24 hilos: para cada uno, la diferencia máxima frente a la referencia y el checksum. **Esta tabla es obligatoria y va antes que cualquier tabla de tiempos de la memoria.**

- **[PROG] Fallo provocado.** Escribid a propósito una versión que actualice **in situ** (un solo buffer), comprobad que la verificación la rechaza, y mostrad la evidencia de que su resultado **cambia con el número de hilos**. Es la prueba de que vuestro protocolo sirve para algo.
- **[RAZ]** Justificad la elección del bucle: por qué el de filas y no el de columnas, y qué relación tiene con cómo está almacenada la malla.
- **[RAZ]** Justificad cada cláusula de datos: qué es compartido, qué es privado y por qué. Incluid qué error concreto se produciría si cada una estuviera mal.
- **[RAZ]** Declarad y **justificad la tolerancia** de la verificación. Explicad por qué el campo puede salir idéntico bit a bit y en cambio el residuo o el checksum pueden no salirlo, y qué tiene eso que ver con la asociatividad de la suma en coma flotante.
- **[RAZ]** Explicad qué barreras hay en vuestro código, cuáles son implícitas, cuáles son imprescindibles y por qué no se puede quitar la que separa dos pasos de tiempo.

Ejercicio 3 (2,0 puntos) — Escalabilidad fuerte

Medid cómo se acorta el reloj al añadir hilos sobre un problema de tamaño fijo, y explicad la curva que salga.

- **[PROG]** Barrido completo con **1, 2, 4, 8, 12, 16, 20 y 24 hilos**, malla de 2048×2048 y 300 pasos. Fijad el número de hilos con `omp_set_num_threads()` o con `OMP_NUM_THREADS`, y desactivad el ajuste dinámico con `omp_set_dynamic(0)` para que el número que pedís sea el que se usa.
- **[PROG]** Tabla con tiempo, **speedup** y **eficiencia** para cada recuento de hilos. Indicad repeticiones, criterio (mínimo o media) y máquina.
- **[PROG]** Declarad si el speedup es **absoluto** (frente a la secuencial compilada sin `-fopenmp`) o **relativo** (frente a la paralela con un hilo). Se pide el absoluto; si dais los dos, etiquetadlos.
- **[PROG] Medid el techo de vuestra máquina** con un experimento propio del tipo del apartado IV.C, y dibujadlo en la misma gráfica que vuestra curva, junto a la diagonal ideal. Si trabajáis en la máquina de la asignatura podéis contrastar con la tabla del enunciado, pero la medida tiene que ser vuestra.
- **[PROG]** Repetid el barrido con una malla **pequeña** (por ejemplo $N = 256$), **ajustando el número de pasos** para que el trabajo total sea comparable al de la malla grande, y comparad las dos curvas de speedup. Aviso: el resultado puede sorprenderos, y si os sorprende, ese es el ejercicio.
- **[RAZ]** Identificad los **tramos** de vuestra curva y explicad cada uno **por separado**: el tramo en que el speedup crece, el punto en que deja de crecer y la meseta o el descenso posterior. Son fenómenos distintos, hay que nombrarlos y hay que decir **en qué número de hilos** está cada frontera, porque ese número es un dato sobre vuestra máquina.
- **[RAZ]** Si la curva **baja** al añadir hilos —que es un resultado posible y frecuente en este problema— explicad el mecanismo. Añadir hilos a un recurso ya saturado no es neutro: cuesta. Decid qué recurso se ha saturado y qué es lo que cuesta.
- **[RAZ]** Explicad por qué la malla pequeña escala distinto que la grande. Hay **dos** mecanismos candidatos y van en sentidos opuestos: por un lado, una malla pequeña cabe en la caché y deja de depender de la memoria principal, lo que *debería* ayudar; por otro, deja tan poco trabajo en cada paso que el coste fijo de repartirlo y sincronizarlo pesa mucho más, lo que estorba. **Decid cuál domina en vuestras medidas y con qué evidencia**. Una forma de separarlos: medid el tiempo por paso y comparadlo con el coste de una barrera.
- **[RAZ]** Estimad qué fracción del programa es efectivamente secuencial y contrastadla con la ley de Amdahl. Decid honestamente hasta qué punto Amdahl explica o no explica vuestra curva: en un problema limitado por el ancho de banda de memoria, el reparto en «parte serie» y «parte paralela» es una simplificación, y detectar sus límites vale más que aplicar la fórmula a ciegas.

Ejercicio 4 (1,0 punto) — Escalabilidad débil

- **[PROG]** Barrido con los mismos ocho recuentos de hilos, haciendo crecer la malla como $N(p) = N_1 \cdot \sqrt{p}$ desde $N_1 = 1024$, con el número de pasos fijo. Indicad los N exactos que habéis usado tras redondear.
- **[PROG]** Tabla y gráfica de tiempo frente a número de hilos, con la línea plana ideal de referencia. Calculad también la **eficiencia débil**, T_1/T_p.
- **[RAZ]** Justificad la raíz cuadrada: por qué N se multiplica por $\sqrt{p}$ y no por p.
- **[RAZ]** Comparad las dos escalabilidades sobre vuestros propios datos: qué pregunta responde cada una, cuál sale mejor en TermoHPC y por qué. Si a alguien le sale la débil mucho mejor que la fuerte, la explicación es interesante y hay que darla.
- **[RAZ]** Al crecer la malla crece también la memoria: con $N = 5017$ los dos buffers ocupan del orden de 400 MB. Comentad si eso ha afectado a vuestras medidas y cómo lo habéis comprobado.

Ejercicio 5 (1,5 puntos) — Comparación de planificaciones

- **[PROG]** Medid, con 12 hilos y la configuración principal, al menos: `static` sin trozo, `static` con trozos de 1, 4, 16 y 64 filas, `dynamic` con trozos de 1, 4, 16 y 64, y `guided`. Tabla completa y gráfica de barras.
- **[PROG]** Verificad que **todas** las planificaciones dan el mismo campo. Si alguna no lo da, el problema no es la planificación: es que hay una dependencia que no habíais visto, y encontrarla es más valioso que la tabla entera.
- **[RAZ]** Justificad cuál elegís y por qué, **a partir de vuestros datos**. Citar la regla general sin medirla no puntúa.
- **[RAZ]** Explicad por qué `dynamic` con trozos pequeños sale especialmente mal en este problema. Hay dos motivos distintos —el coste de gestión y la pérdida de localidad— y hay que separarlos.
- **[RAZ]** Explicad qué le pasa a `static` con trozo de 1 fila frente a `static` sin trozo, que también son dos repartos estáticos y sin embargo no cuestan lo mismo.
- **[RAZ]** Argumentad en qué condiciones cambiaría vuestra elección: describid una modificación del problema —no del código— que haría ganar al reparto dinámico.

Ejercicio 6 (2,0 puntos) — Optimización guiada por medición

El ejercicio de más peso, y el que mejor distingue a un grupo que ha entendido la asignatura. Se pide **al menos dos iteraciones completas** del ciclo: medir, formular una hipótesis sobre el cuello de botella, cambiar **una sola cosa**, volver a medir, y concluir si la hipótesis era correcta.

- **[PROG] Punto de partida documentado**: la versión del Ejercicio 2, con su tiempo y su speedup.
- **[PROG] Iteración 1**, completa: la medida que motiva la hipótesis, la hipótesis enunciada antes de tocar nada, el cambio (uno solo), la nueva medida y la conclusión.
- **[PROG] Iteración 2**, con la misma estructura, partiendo de donde dejó la primera.
- **[PROG]** Verificación numérica **después de cada iteración**. Una optimización que cambia el resultado no es una optimización.
- **[PROG]** Tabla resumen con todas las variantes medidas, su tiempo, su speedup y la mejora que aportó cada una respecto de la anterior.
- **[RAZ]** Para cada iteración, la **evidencia** que sostiene la hipótesis. «Nos pareció que…» no es evidencia; «al pasar de 300 a 30 pasos el tiempo por paso sube un 40 %, luego hay un coste fijo por paso» sí lo es.

- **[RAZ]** Al menos **una iteración que no funcionó**: una hipótesis razonable que la medida no confirmó, con la explicación de por qué. Esto se pide explícitamente y puntúa; una memoria en la que todo salió bien a la primera no resulta creíble.
- **[RAZ]** Estimad el **ancho de banda de memoria** que consigue vuestro programa: bytes movidos por paso divididos entre el tiempo por paso. Comparadlo con el máximo teórico de vuestra máquina y decid qué fracción alcanzáis. Es la prueba objetiva de si el problema está limitado por la memoria, y es el argumento más sólido que podéis dar en la defensa.

Candidatos razonables para las iteraciones —no hace falta probarlos todos, y no todos mejoran—: sacar la región paralela fuera del bucle temporal; calcular el residuo cada k pasos en lugar de en todos; probar `collapse(2)`; fijar la afinidad de los hilos con `OMP_PROC_BIND` y `OMP_PLACES`; inicializar la malla en paralelo con el mismo reparto que usa el cálculo; o comprobar si el compilador está vectorizando el bucle interior con `-fopt-info-vec`.

> **La regla del cambio único**
>
> Cada iteración cambia **una cosa**. Si se cambian tres a la vez y el tiempo mejora, no se sabe cuál de las tres lo hizo —ni si alguna de ellas lo empeoró—. Una memoria que presenta «la versión optimizada» con cinco cambios simultáneos y un único número al final no demuestra método, y así se califica.

RETO (OPCIONAL, HASTA +1,0 PUNTO SOBRE LA NOTA FINAL, CON TOPE EN 10)

Elegid **uno** de los dos y desarrolladlo con el mismo rigor que el resto:

Reto A · Jacobi frente a actualización in situ. El Ejercicio 2 pedía provocar la versión in situ para comprobar que la verificación la rechaza. Aquí se le da la vuelta: esa versión no es «Jacobi mal hecho», es el **método de Gauss-Seidel**, que es otro método legítimo y que **converge en menos pasos**. Implementadlo, medid cuántos pasos necesita cada uno para alcanzar la misma tolerancia, y discutid el compromiso completo: Gauss-Seidel hace menos pasos pero cada paso es más difícil de paralelizar y su resultado depende del reparto. ¿Compensa? Justificadlo con tiempos totales hasta convergencia, no con tiempos por paso. Como extra, investigad el recorrido **rojo-negro**, que recupera el paralelismo dividiendo la malla en dos subconjuntos independientes.

Reto B · Bloqueo por bloques. Implementad el recorrido por bloques del apartado V.F, barred el tamaño de bloque (16, 32, 64, 128, 256) y explicad la curva que salga, relacionando el tamaño de bloque con el de las cachés de vuestra máquina. Atención: **este reto se puntúa por el análisis, no por la mejora**. Es perfectamente posible que el bloqueo **no** mejore nada en vuestra malla, o que la empeore; si es así, hay que decirlo y explicar por qué —el bloqueo mejora la reutilización pero rompe el recorrido secuencial de la memoria, y los dos efectos van en sentidos contrarios—. Una memoria que mide, no mejora y lo argumenta bien puntúa más que otra que presenta una mejora sin explicarla. Para reforzar el análisis, repetid el barrido con una malla mayor y comprobad si la conclusión cambia con el tamaño.

Plan de trabajo sugerido

El laboratorio es de varias semanas y de cinco o seis personas, y ahí está la primera dificultad real: **es muy fácil que tres personas midan y dos miren**. Este reparto no es obligatorio, pero funciona y evita el problema clásico de que todo el mundo espere a que esté el código para poder empezar.

FASE	QUÉ SE HACE	SE PUEDE HACER EN PARALELO CON…
1. Arranque Ejercicio 1	La versión secuencial correcta, modular y con las comprobaciones físicas. Hasta que esto no esté, no hay referencia y no se puede verificar nada.	Montar el banco de pruebas: los guiones que lanzan los barridos y recogen las salidas en tablas. No hace falta que el programa esté para escribirlos.
2. Paralelización Ejercicio 2	La directiva, las cláusulas y —sobre todo— el protocolo de verificación con el fallo provocado.	Medir el techo de cálculo y el de ancho de banda de vuestra máquina. Son independientes del programa y hacen falta para interpretarlo todo.
3. Medidas Ejercicios 3, 4 y 5	Los tres barridos: escalabilidad fuerte, débil y planificaciones. Es la fase que más tiempo de máquina consume y la que mejor se reparte.	Redactar las secciones de fundamentos y de justificación de la memoria, que no dependen de los números finales.
4. Optimización Ejercicio 6	Las dos iteraciones. Es la fase que **no se puede comprimir**: cada iteración necesita medir, pensar y volver a medir.	El reto, si lo hacéis. Y la revisión de que cada tabla de la memoria se puede reproducir con el código que vais a pegar.
5. Cierre	Reflexión, repaso de la lista de comprobación de esta sección y ensayo de la defensa.	—

DOS CONSEJOS QUE SALEN DE VER MUCHOS DE ESTOS LABORATORIOS

Automatizad los barridos desde el primer día. Un guion que lanza las ocho ejecuciones y escribe la tabla se escribe en media hora y se usa cincuenta veces. Medir a mano garantiza que, cuando encontréis un error a mitad de la fase 4, no repitáis las medidas y acabéis entregando tablas de versiones distintas del programa.

Guardad las salidas en bruto, con la fecha y el número de hilos, en un fichero por barrido. Es lo que os permitirá responder en la defensa a «¿de dónde sale este 2,3?», y es lo que distingue una medida de una cifra.

VIII. La memoria: contenido y lista de comprobación

El entregable es una memoria que **razona**, no un informe que muestra. Las listas siguientes son la lista de comprobación con la que se corrige: **si algo no aparece en la memoria, no se evalúa**, por muy bien que esté en vuestro código.

A. Entregables de programación y medida

- El código completo —secuencial, paralelo, verificación y variantes de optimización— **pegado dentro del PDF como texto**, comentado y legible, compilado sin avisos con `-Wall -Wextra -O2 -fopenmp`.
- La tabla de verificación con los ocho recuentos de hilos, **antes** de cualquier tabla de tiempos.
- La evidencia del fallo provocado del Ejercicio 2: la versión in situ rechazada y su dependencia del número de hilos.
- Las tablas de escalabilidad fuerte (tiempo, speedup, eficiencia) y débil (tiempo, eficiencia débil), con repeticiones, criterio y máquina declarados.
- La medida propia del techo de la máquina y su línea dibujada en la gráfica de speedup, junto a la diagonal ideal.
- Las dos curvas de speedup, malla grande y malla pequeña.
- La tabla y la gráfica de la comparación de planificaciones, con la verificación de que todas dan el mismo campo.
- Las dos iteraciones de optimización completas, cada una con su medida de partida, su hipótesis, su cambio único, su nueva medida y su verificación.
- La iteración que no funcionó.
- La estimación del ancho de banda alcanzado y su comparación con el máximo de la máquina.
- Las órdenes exactas de compilación y ejecución, para que cualquier tabla se pueda reproducir.

B. Entregables de razonamiento

- Por qué el bucle temporal es secuencial, enunciado como dependencia entre iteraciones.
- Por qué se paraleliza el bucle de filas y no el de columnas, ligado a cómo está almacenada la malla.
- La justificación de cada cláusula de datos, con el error concreto que provocaría cada una mal puesta.
- La tolerancia de la verificación, declarada y justificada, y la explicación de por qué el campo puede salir idéntico bit a bit y las reducciones no.
- La lectura por tramos de la curva de speedup (1–4, 4–12, 12–24), **frente al techo medido** y no frente a la diagonal.
- Por qué la malla pequeña escala distinto que la grande, situando cada una en la jerarquía de memoria.
- El análisis de Amdahl y, sobre todo, el reconocimiento honesto de hasta dónde llega y dónde deja de explicar vuestros datos.
- La justificación de la raíz cuadrada en la escalabilidad débil y la comparación de las dos escalabilidades.
- La elección de planificación argumentada con vuestros datos, y la condición bajo la cual cambiaría.
- El argumento completo de por qué este problema está —o no— limitado por el ancho de banda de memoria, apoyado en la medida del ancho de banda.

C. Conclusiones y reflexión del grupo

Una reflexión **auténtica y específica**, que no es un resumen. Debe recoger:

- Qué se ha aprendido, y en qué momento concreto se aprendió (qué medida os cambió la idea que teníais).
- Qué dificultades surgieron y cómo se resolvieron.
- Qué decisiones de diseño se tomaron y por qué: el tamaño de malla, el criterio de medida, el número de repeticiones, el trozo de la planificación.
- **Dónde lo medido coincidió con lo esperado y dónde no**, y qué explica la diferencia. Esto es lo que más peso tiene.
- Qué esperáis que cambie en el Laboratorio 2 al pasar este mismo problema a MPI y a la GPU, y por qué. No hace falta acertar; hace falta razonar.
- El **reparto del trabajo** dentro del grupo.

Una reflexión que podría servir para cualquier laboratorio de cualquier asignatura se considera insuficiente.

> **La defensa**
>
> Es **obligatoria** y **todos los integrantes intervienen**. Se podrá pedir modificar el programa en el momento, rehacer una medida con otros parámetros, explicar una línea concreta del código o justificar un número de una tabla. La incapacidad de razonar las decisiones durante la defensa afecta a la calificación de todo el trabajo, no solo de la parte preguntada.

IX. Rúbrica de evaluación

La calificación (0–10) se obtiene aplicando la siguiente rúbrica analítica sobre el conjunto del laboratorio. Cada criterio se valora en cuatro niveles —**Excelente** (100), **Notable** (75), **Aceptable** (50), **Insuficiente** (≤25)— y aporta su peso a la nota final. Está diseñada para que **una entrega que funcione pero no razone no alcance el aprobado**.

CRITERIO · PESO	NIVELES DE DESEMPEÑO
1. Versión secuencial de referencia **Peso 12 %**	**Exc.** Programa modular y parametrizado, doble buffer con intercambio de punteros, los dos modos de parada, instrumentación completa y las tres comprobaciones físicas (bordes, residuo decreciente, rango). Contrasta el crecimiento con N^2. **Not.** Correcto con algún detalle menor: falta una comprobación física o el contraste con N^2 es superficial. **Ace.** Funciona, pero es monolítico, o copia la malla en vez de intercambiar punteros, o no está instrumentado. **Ins.** No simula correctamente, o no hay versión secuencial de referencia.
2. Paralelización y justificación de las decisiones **Peso 16 %**	**Exc.** Paraleliza el bucle correcto con `default(none)` y `reduction(max)`; justifica la elección del bucle, cada cláusula de datos y cada barrera, con el error concreto que provocaría cada fallo. **Not.** Paralelización correcta con alguna justificación floja o alguna cláusula sin razonar. **Ace.** Funciona, pero las decisiones se citan sin justificar, o no usa `default(none)`. **Ins.** No paraleliza correctamente, o las cláusulas son incorrectas.
3. Verificación numérica **Peso 12 %**	**Exc.** Protocolo con las dos magnitudes, tabla con los ocho recuentos de hilos, tolerancia declarada y justificada, y el fallo provocado in situ demostrando que la verificación lo rechaza y que su resultado depende del número de hilos. **Not.** Verificación correcta y completa, pero la tolerancia no se justifica o el fallo provocado está poco desarrollado. **Ace.** Compara solo el checksum, o solo con un número de hilos, o no provoca ningún fallo. **Ins.** Sin verificación, o con resultados que no coinciden con la referencia. **En este nivel no se evalúan los criterios 4 a 7.**
4. Escalabilidad fuerte: medida e interpretación **Peso 16 %**	**Exc.** Barrido completo con condiciones declaradas; speedup y eficiencia bien calculados y etiquetados como absolutos; techo de la máquina medido por el grupo y dibujado; lectura por tramos que explica los tres regímenes; contraste malla grande frente a malla pequeña con la explicación de memoria. **Not.** Barrido y gráficas correctos, con la interpretación algo incompleta o sin el contraste de tamaños. **Ace.** Mide y grafica, pero interpreta comparando solo con la diagonal ideal, o no distingue speedup absoluto de relativo. **Ins.** Sin barrido completo, o tablas no reproducibles con el código entregado.
5. Escalabilidad débil **Peso 8 %**	**Exc.** Barrido con N creciendo como $\sqrt{p}$, justificación de la raíz, eficiencia débil, gráfica con la referencia plana y comparación razonada con la fuerte. **Not.** Estudio correcto con la comparación entre las dos escalabilidades algo genérica. **Ace.** Hace crecer la malla mal (linealmente con p), o mide sin interpretar. **Ins.** Ausente.
6. Comparación de planificaciones **Peso 10 %**	**Exc.** Mide todas las combinaciones pedidas, verifica que todas dan el mismo campo, justifica la elección con sus datos, separa las dos causas del mal comportamiento de `dynamic` y describe cuándo cambiaría la elección. **Not.** Comparación completa con alguna explicación poco desarrollada. **Ace.** Mide algunas planificaciones y elige, pero cita la regla general en vez de razonar sobre sus propios datos. **Ins.** Ausente, o sin medidas que sostengan la elección.

CRITERIO · PESO	NIVELES DE DESEMPEÑO
7. Optimización guiada por medición **Peso 14 %**	**Exc.** Dos iteraciones completas con evidencia previa, hipótesis, un solo cambio, nueva medida y verificación; incluye una iteración que no funcionó y la explica; estima el ancho de banda alcanzado y lo compara con el de la máquina. **Not.** Dos iteraciones correctas, pero sin la iteración fallida o sin la estimación del ancho de banda. **Ace.** Optimiza y mejora, pero cambia varias cosas a la vez o no documenta la evidencia que motivó cada cambio. **Ins.** Sin optimización, o presentada como una única versión final sin proceso.
8. Memoria, razonamiento y presentación **Peso 12 %**	**Exc.** Memoria que razona y conecta decisiones, medidas y explicaciones; reflexión específica del grupo con discrepancias propias; código pegado como texto, comentado y legible; gráficas rotuladas; ortografía cuidada. **Not.** Memoria completa y legible, con la reflexión algo genérica o detalles de presentación mejorables. **Ace.** Desordenada, o la reflexión es un resumen, o las gráficas no están rotuladas. **Ins.** Difícilmente legible, o el código va como capturas de pantalla.

El RETO de la sección VII suma hasta **1,0 punto adicional** sobre la nota resultante, con el límite de 10. No compensa un criterio insuficiente: si la verificación no pasa, el reto no se corrige.

X. Qué invalida o penaliza la entrega

Lo que sigue no son avisos genéricos: son las situaciones concretas que anulan trabajo. Leedlas antes de entregar.

A. Lo que invalida

- **Resultados que no coinciden con la referencia secuencial.** Si la verificación no pasa y no se explica, **los ejercicios 3 a 6 no se evalúan**: sus tablas miden un programa que calcula otra cosa. No hay excepción por «funciona casi siempre».
- **Tablas de tiempos no reproducibles con el código entregado.** Si los números no se pueden obtener ejecutando lo que hay pegado en el PDF, esa tabla no cuenta como evidencia y se abre revisión de autoría.
- **Ausencia de la tabla de verificación** del Ejercicio 2. Es la condición de entrada de todo lo demás.
- **Medidas sin declarar las condiciones**: sin número de repeticiones, sin criterio (mínimo o media), sin tamaño de malla, sin número de pasos o sin máquina. Una tabla así no se puede interpretar y no se puntúa.
- **Comparar tiempos con distinto número de pasos** o distinto tamaño de malla en la misma tabla de speedup.

B. Lo que penaliza

- **Código presentado como capturas de pantalla.** Penaliza la presentación y puede impedir evaluar el ejercicio afectado, porque no se puede leer ni comprobar.
- **Afirmaciones de rendimiento sin datos.** «Mejora bastante», «es mucho más rápido», «la sobrecarga es despreciable»: cada una de esas frases necesita un número al lado o se tacha.
- **Culpar a la sobrecarga de OpenMP sin haberla medido.** Es la explicación comodín de este laboratorio y casi nunca es la correcta. Si la afirmáis, medidla.
- **Presentar el speedup frente a la diagonal ideal sin el techo real**, o disculparse por no alcanzar un ideal que la máquina no permite. Se penaliza igual que exagerar en el otro sentido: las dos cosas son errores de análisis.
- **Dar speedup relativo llamándolo speedup** sin decir que la referencia es la versión paralela con un hilo.
- **Optimizaciones con varios cambios simultáneos** presentadas como una sola iteración.
- **Medir con** `clock()` en lugar de con el reloj de pared, o medir sin `-O2`, o cronometrar incluyendo la inicialización.
- **Un ejercicio correcto pero sin justificar**: como máximo, la mitad de su puntuación.

C. Indicadores de uso indebido de IA

- Tablas de tiempos que no se corresponden con el hardware declarado o con el código entregado; o speedups superiores al techo físico de la máquina sin explicación.
- Curvas de speedup sospechosamente limpias, sin ninguna irregularidad, en una máquina de núcleos heterogéneos.
- Explicaciones genéricas de escalabilidad que valdrían para cualquier programa, sin conexión con la implementación concreta del grupo ni con sus números.
- Código que no compila pese a presentarse como funcional, o cláusulas de datos que no se corresponden con el análisis que la memoria afirma haber hecho.
- Una memoria sin ninguna discrepancia, sin ninguna hipótesis fallida y sin ninguna decisión propia.
- Incapacidad de razonar una decisión, rehacer una medida o explicar una línea del código durante la defensa.

XI. Normativa de entrega

A. Formato y nombre del archivo

> **UN ÚNICO PDF POR EQUIPO**
>
> Se entrega **un solo archivo PDF** en el Aula Virtual, con toda la memoria dentro. **No se entrega código suelto, ni un archivo comprimido, ni un enlace a un repositorio.** Lo que no esté dentro del PDF no se corrige.

Nombre del archivo: `CAR_Lab1_NombreGrupo.pdf` (por ejemplo, `CAR_Lab1_JacobiNoDuerme.pdf`).

Dentro del PDF va **todo**:

- La memoria completa, organizada ejercicio por ejercicio.
- El **código pegado como texto**, no adjunto y no como fotografía de la pantalla.
- Las **capturas de los resultados**: la salida por terminal de las ejecuciones que sostienen cada tabla.
- Las tablas de tiempos, speedup y eficiencia.
- Las gráficas, con ejes rotulados y unidades.
- Las observaciones del equipo y la reflexión final.

B. Contenido mínimo

- **Portada**: nombre y apellidos de todos los integrantes, nombre del grupo, título del laboratorio y fecha.
- **Entorno**: máquina (modelo de procesador, núcleos físicos y lógicos, memoria), sistema operativo, versión del compilador y órdenes exactas de compilación.
- **Objetivos**: qué se pretende, en pocas líneas.
- **Desarrollo por ejercicio**: implementación (código como texto), justificación y evidencia.
- **Conclusiones y reflexión** del grupo, con el reparto del trabajo.
- **Bibliografía**, citada correctamente.

Se cuidará la presentación y la ortografía. La defensa es obligatoria.

XII. Criterios académicos

A. Sistema de calificaciones

0–4,9 Suspenso; 5,0–6,9 Aprobado; 7,0–8,9 Notable; 9,0–10 Sobresaliente. La mención de «Matrícula de Honor» podrá otorgarse a quienes obtengan 9,0 o más, con el límite reglamentario aplicable.

B. Peso del laboratorio

Este laboratorio forma parte del bloque de actividades y proyecto de la asignatura, según los pesos fijados en la guía docente. Se califica de 0 a 10 aplicando la rúbrica de la sección IX. Es **imprescindible entregarlo** para poder superar la asignatura.

C. Plazo y «No Presentado»

El plazo de entrega y la fecha de defensa se publicarán en el Aula Virtual. La no entrega en el plazo fijado implica un «No Presentado» en el laboratorio, que deberá recuperarse según las instrucciones de la convocatoria extraordinaria.

D. Defensa y autoría

La defensa es obligatoria y **todos los integrantes deben intervenir**. El profesor podrá pedir modificar el programa en el momento —cambiar la planificación, alterar el tamaño de la malla, rehacer una medida— y el grupo deberá demostrar que conoce su código y sabe ejecutarlo.

E. Advertencia sobre plagio

La Universidad Francisco de Vitoria no tolera el plagio ni la copia. Cualquier intento de defraudar el sistema de evaluación —copiar código, medidas o análisis de otros compañeros, de Internet o de materiales no autorizados sin citar la fuente— se sancionará según la Normativa de Convivencia de la UFV. **Copiar una tabla de tiempos es especialmente grave**, porque una medida es un hecho sobre una máquina concreta y en una fecha concreta: presentarla como propia es inventar un resultado experimental.

F. Uso de inteligencia artificial generativa

El uso de IA generativa debe ser transparente y ajustarse a las indicaciones del profesor. En este laboratorio, el diseño, la implementación, **las medidas**, el análisis y la memoria deben ser de elaboración propia del grupo; la autoría se verifica en la defensa oral. Las medidas, en particular, no se pueden delegar: solo se obtienen ejecutando el programa en una máquina real.

G. Bibliografía recomendada

- Chapman, Jost, van der Pas — *Using OpenMP: Portable Shared Memory Parallel Programming*, MIT Press, 2007.
- Quinn — *Parallel Programming in C with MPI and OpenMP*, McGraw-Hill, 2003.
- Kumar, Grama, Gupta, Karypis — *Introduction to Parallel Computing*, 2.ª ed., Addison-Wesley, 2003. Capítulos 5 (métricas de rendimiento) y 13 (métodos de discretización).
- Especificación de OpenMP, disponible en `openmp.org`. La lista de cláusulas de `schedule` y la semántica de las barreras implícitas están ahí, y es la fuente que hay que citar.